



ĐẠI HỌC QUỐC GIA THÀNH PHỐ HỒ CHÍ MINH
TRƯỜNG ĐẠI HỌC CÔNG NGHỆ THÔNG TIN

LẬP TRÌNH PYTHON CHO MÁY HỌC - CS116.P22

Báo cáo Cuối kỳ DỰ ĐOÁN DUYỆT VAY

Nhóm sinh viên :

Huỳnh Việt Tiến - 23521570
Nguyễn Công Phát - 23521143
Nguyễn Minh Bảo - 23520123
Phạm Phú Bảo - 23520133

GVHD :

TS. Nguyễn Vinh Tiệp

Ngày 29 tháng 6 năm 2025

Mục lục

LỜI NÓI ĐẦU	3
1 GIỚI THIỆU BÀI TOÁN	4
2 MÔ TẢ DATASET	5
2.1 Tổng quan Dataset	5
2.2 Phân tích dữ liệu (EDA)	6
2.2.1 Đọc dữ liệu	6
2.2.2 Data Visualization: Single Variable Analysis	7
2.2.3 Data Visualization: Bivariate Analysis	11
2.2.4 Phân tích phân phối theo các biến phân loại	12
2.2.5 Data Visualization: Multivariate Analysis	13
2.2.6 Kết luận cho giai đoạn EDA	14
3 TỔNG QUAN HƯỚNG TIẾP CẬN VÀ PHƯƠNG PHÁP	16
3.1 Các mô hình học máy sử dụng	16
3.2 Metric đánh giá mô hình	16
3.3 Feature Engineering và xử lý dữ liệu	17
3.4 Kết hợp mô hình và đánh giá	17
4 THỰC NGHIỆM	18
4.1 Pipeline	18
4.2 Tiền xử lý dữ liệu	19
4.2.1 Các chiến thuật xử lý NULL	19
4.2.2 Các chiến thuật xử lý outlier	20
4.2.3 Mã hoá Dữ liệu (Encoding)	25
4.2.4 Chuẩn hoá dữ liệu - Data Scaling	27
4.3 Feature Engineering	31
4.3.1 Loại bỏ các đặc trưng không nhiều giá trị cho việc dự đoán	31
4.3.2 Tạo đặc trưng mới	32
4.3.3 Lựa chọn đặc trưng	33
4.3.4 Kết luận	36
4.4 Huấn luyện, đánh giá và tinh chỉnh mô hình	37
4.4.1 Xác định Metric để evaluate model	37
4.4.2 Chọn Baseline Model	37
4.4.3 Lựa Chọn Tiêu Chí và Mô Hình Phù Hợp	37
4.4.4 Phương pháp tìm các siêu tham số tốt nhất cho các model	38
4.5 Ensemble Model	42



4.5.1	Check Độ Tương Đồng Các Bộ Siêu Tham Số Bằng Metric Euclidean .	42
4.5.2	Train ensemble model	44
4.6	Kết quả thực nghiệm	46
5	KẾT LUẬN VÀ HƯỚNG PHÁT TRIỂN	48
5.1	Kết luận	48
5.2	Hướng phát triển	48
6	BẢNG PHÂN CÔNG CÔNG VIỆC	50

LỜI NÓI ĐẦU

Lời đầu tiên, nhóm chúng tôi xin bày tỏ lòng biết ơn sâu sắc đến thầy **TS. Nguyễn Vinh Tiệp**, người đã tận tình hướng dẫn, định hướng và truyền đạt những kiến thức quý báu trong suốt quá trình thực hiện đồ án này. Sự chỉ bảo tận tâm, những ý kiến đóng góp sâu sắc và sự khích lệ kịp thời của thầy đã là nguồn động lực to lớn, giúp chúng tôi vượt qua những khó khăn và hoàn thành báo cáo này một cách tốt nhất.

Chúng tôi cũng xin chân thành cảm ơn các bạn sinh viên đã nhiệt tình chia sẻ kinh nghiệm học tập, hỗ trợ thảo luận và đóng góp những ý kiến quý giá trong suốt quá trình thực hiện dự án.

Cuối cùng, nhóm chúng tôi xin gửi lời cảm ơn chân thành đến tất cả những ai đã đồng hành và hỗ trợ chúng tôi trong suốt quá trình học tập và nghiên cứu. Sự hỗ trợ và đóng góp của quý thầy cô và các bạn là yếu tố then chốt, quyết định sự thành công của đồ án này.

Chúng tôi vô cùng trân trọng và biết ơn những tình cảm và sự giúp đỡ quý báu đó! Rất mong nhận được sự góp ý từ quý thầy cô để nhóm có thể tiếp tục hoàn thiện và phát triển sản phẩm tốt hơn trong tương lai.

Chương 1

GIỚI THIỆU BÀI TOÁN

Ngày nay, việc vay vốn từ ngân hàng đã trở thành một điều hết sức phổ biến. Các ngân hàng kiếm lợi nhuận từ lãi suất của các khoản vay mà họ cấp cho khách hàng. Khi xem xét phê duyệt một khoản vay, ngân hàng cần cân nhắc đến nhiều yếu tố quan trọng như lịch sử tín dụng, tuổi của người vay, tổng thu nhập hàng năm, tình trạng sở hữu nhà ở (ví dụ: sở hữu, thuê, thế chấp,...) và số năm kinh nghiệm làm việc hiện tại của họ.

Có rất nhiều người nộp đơn xin vay vốn với đa dạng các mục đích như chi trả cho giáo dục, chi phí y tế, đầu tư mạo hiểm, chi tiêu cá nhân và nhiều mục đích khác nữa. Tuy nhiên, không phải tất cả các đơn xin vay đều được phê duyệt dựa trên những điều kiện đã được đề cập ở trên. Thực tế cho thấy, có vô số trường hợp đơn xin vay của người nộp đơn không được các công ty tài chính chấp thuận. Chính vì vậy, việc dự đoán một cách chính xác liệu có nên cấp khoản vay cho một khách hàng hay không đóng vai trò vô cùng quan trọng đối với các ngân hàng để tối đa hóa lợi nhuận và giảm thiểu rủi ro.

Các hình thức cho vay hiện nay rất đa dạng. Thông thường, các khoản vay này được cung cấp bởi các ngân hàng và nhiều tổ chức tài chính khác. Các điều khoản cụ thể của mỗi khoản vay, ví dụ như mức lãi suất và các loại phí liên quan, sẽ có sự khác biệt tùy thuộc vào chính sách của từng bên cho vay và đặc điểm của từng loại hình khoản vay.

Mục tiêu cốt lõi của dự án này là xây dựng một mô hình dự đoán có khả năng phân loại chính xác các đơn xin vay thành hai nhóm: được phê duyệt hoặc bị từ chối. Mô hình này sẽ là một công cụ đắc lực giúp các tổ chức cho vay đưa ra những quyết định sáng suốt trong việc xét duyệt hồ sơ vay vốn, dựa trên đánh giá chính xác về khả năng trả nợ của người nộp đơn. Việc ứng dụng mô hình này không chỉ mở ra cơ hội tiếp cận nguồn vốn cho một lượng lớn khách hàng tiềm năng, đặc biệt là những người chưa có lịch sử tín dụng, mà còn góp phần quan trọng vào việc giảm thiểu rủi ro tín dụng cho các nhà cung cấp dịch vụ tài chính.

Trong báo cáo này, nhóm sẽ trình bày chi tiết theo từng bước mà chúng tôi đã áp dụng để giải quyết bài toán.

Chương 2

MÔ TẢ DATASET

2.1 Tổng quan Dataset

Bộ dữ liệu được sử dụng trong bài toán này là bộ dữ liệu được lấy từ một cuộc thi trên Kaggle: Playground Series - Season 4, Episode 10: Loan Approval Prediction Dataset. Bộ dữ liệu này bao gồm một số lượng lớn các bảng (tabular data) và các kiểu dữ liệu đa dạng (int64, float64, object) chứa các thông tin chi tiết về các ứng viên nộp đơn vay vốn:

- Tập huấn luyện: 58,645 mẫu..
- Tập kiểm tra: 39,098 mẫu.
- Số lượng đặc trưng: 13 (bao gồm cả Đặc trưng mục tiêu). Các đặc trưng được chia thành ba loại chính:
 - 8 Đặc trưng số (Numeric features).
 - 4 Đặc trưng phân loại (Categorical features)
 - 1 Đặc trưng mục tiêu.



Feature	Mô tả
id	Mã của người vay (Mã duy nhất cho mỗi khoảng vay)
person_age	Tuổi của người nộp đơn vay
person_income	Tổng thu nhập hàng năm
person_home_ownership	Tình trạng sở hữu nhà (RENT, OWN, MORTGAGE...)
person_emp_length	Số năm đi làm
loan_intent	Mục đích vay
loan_grade	Xếp hạng khoản vay (A-E)
loan_amnt	Số tiền vay
loan_int_rate	Lãi suất vay
loan_percent_income	Tỷ lệ phần trăm thu nhập dùng để trả nợ
cb_person_cred_hist_length	Lịch sử tín dụng (năm)
cb_person_default_on_file	Từng vỡ nợ trong quá khứ (Y/N)
loan_status	Trạng thái vay (0 = Từ chối, 1 = Duyệt)

Bảng 2.1: Mô tả các đặc trưng trong dataset.

2.2 Phân tích dữ liệu (EDA)

2.2.1 Đọc dữ liệu

Phân tích dữ liệu ban đầu

Cấu trúc dữ liệu: Tập huấn luyện bao gồm 58,645 mẫu với 13 đặc trưng, trong đó có 4 cột dạng phân loại (categorical), 3 cột dạng số thực (float), và 6 cột dạng số nguyên (int). Kiểm tra sơ bộ cho thấy **không có giá trị thiếu (missing values)** trong tập dữ liệu, giúp giảm thiểu bước xử lý dữ liệu trước (preprocessing).

Thông kê mô tả: Một số điểm nổi bật từ thống kê mô tả:

- **Tuổi (person_age)** trung bình khoảng 27.55, nhưng có giá trị tối đa là 123 → có thể là outlier.
- **Thu nhập (person_income)** dao động từ 4,200 đến 1,900,000 với trung bình 64,046 → phân phối thu nhập có độ lệch lớn.
- **Số năm làm việc (person_emp_length)** có trung bình khoảng 4.7 năm, nhưng cũng xuất hiện giá trị 123 năm → nghi ngờ outlier.
- **Lãi suất (loan_int_rate)** dao động từ 5.42% đến 23.22%, trung bình 10.68%.
- **Tỷ lệ thu nhập dùng để trả nợ (loan_percent_income)** trung bình là 15.9%, với giá trị tối đa lên đến 83% → dấu hiệu tiềm ẩn rủi ro.
- **Tình trạng vay (loan_status):** tỷ lệ duyệt vay ($\text{loan_status} = 1$) chỉ chiếm khoảng 14.2%, trong khi 85.8% là bị từ chối → vấn đề mất cân bằng nhãn (imbalanced labels).

Phân tích biến phân loại:

- **person_home_ownership:** Gồm các giá trị 'RENT', 'OWN', 'MORTGAGE', 'OTHER'. Đây là biến phân loại rời rạc → sử dụng **one-hot encoding**.



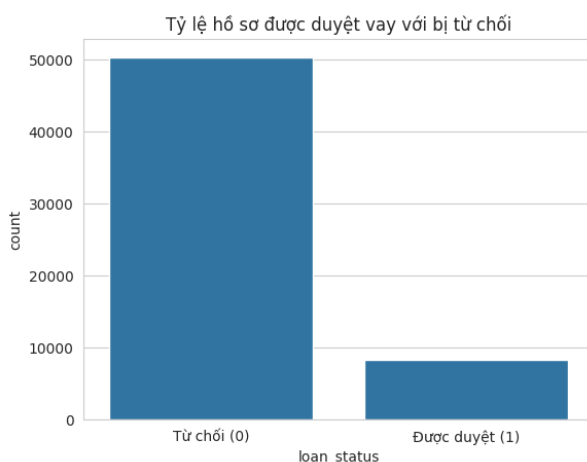
- **loan_intent**: Gồm 6 mục đích vay khác nhau như 'EDUCATION', 'MEDICAL',... → áp dụng **one-hot encoding**.
- **loan_grade**: Gồm các mức 'A' đến 'G', có thứ tự rủi ro tăng dần → sử dụng **label encoding** theo thứ tự (A: tốt nhất → G: tệ nhất).
- **cb_person_default_on_file**: Là biến nhị phân ('Y', 'N') → có thể sử dụng **label encoding** ('Y' = 1, 'N' = 0).

Tóm lại, dữ liệu có cấu trúc tốt, không có giá trị thiếu, và có thể tiến hành tiền xử lý đặc trưng với các kỹ thuật mã hóa phù hợp. Một số giá trị nghi ngờ là ngoại lệ (outliers) cần được xem xét kỹ hơn trong bước xử lý dữ liệu tiếp theo.

2.2.2 Data Visualization: Single Variable Analysis

Phân phối của Đặc trưng mục tiêu

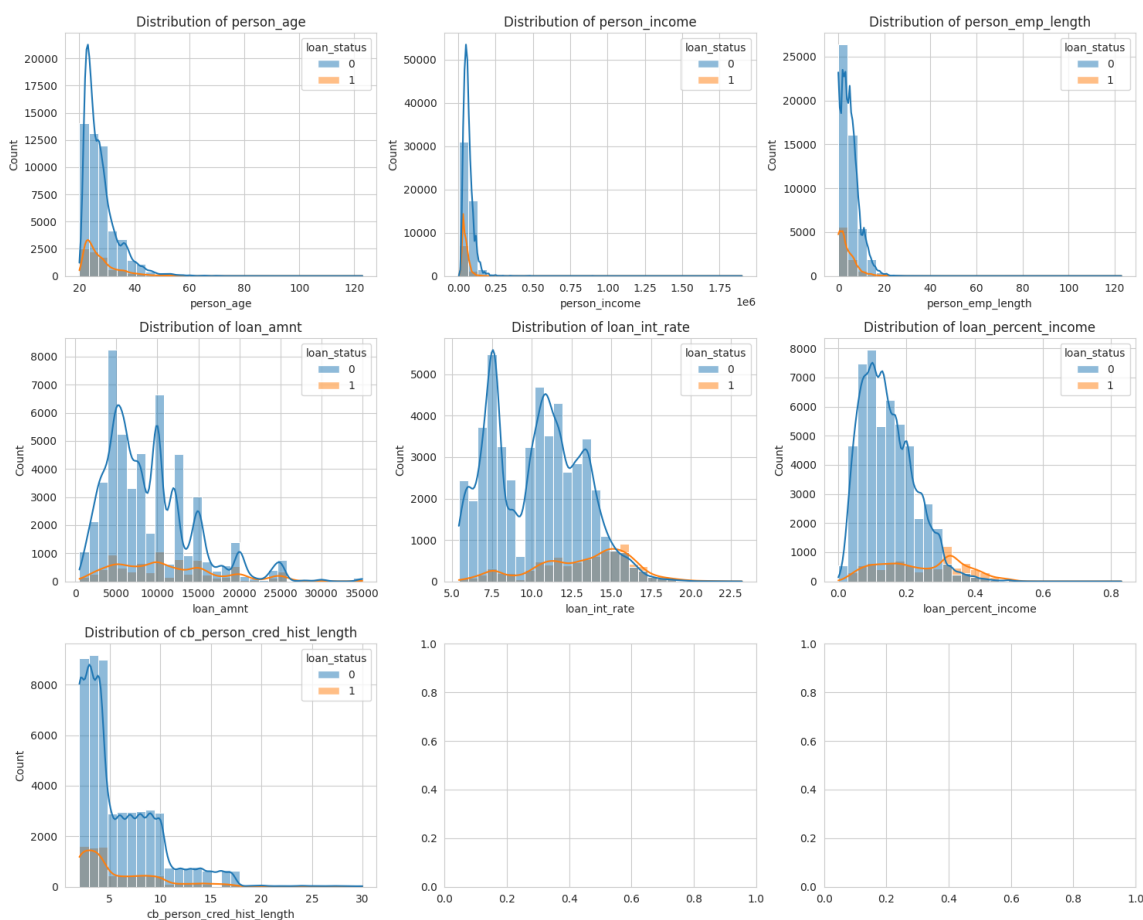
Hình 2.1 cho thấy sự mất cân bằng rõ rệt trong phân phối biến mục tiêu `loan_status`. Tỷ lệ hồ sơ bị từ chối (class 0) chiếm khoảng 86%, trong khi chỉ có 14% hồ sơ được duyệt (class 1). Điều này cho thấy cần sử dụng các kỹ thuật xử lý mất cân bằng dữ liệu trong quá trình xây dựng mô hình.



Hình 2.1: Tỷ lệ hồ sơ được duyệt và bị từ chối

Phân phối của các Đặc trưng số

Qua hình 2.2, chúng ta trực quan hóa phân phối của các đặc trưng số bao gồm: `person_age`, `person_income`, `person_emp_length`, `loan_amnt`, `loan_int_rate`, `loan_percent_income`, và `cb_person_cred_hist_length` thông qua các biểu đồ histogram phân lớp theo biến mục tiêu `loan_status`.



Hình 2.2: Phân phối các đặc trưng số theo trạng thái khoản vay

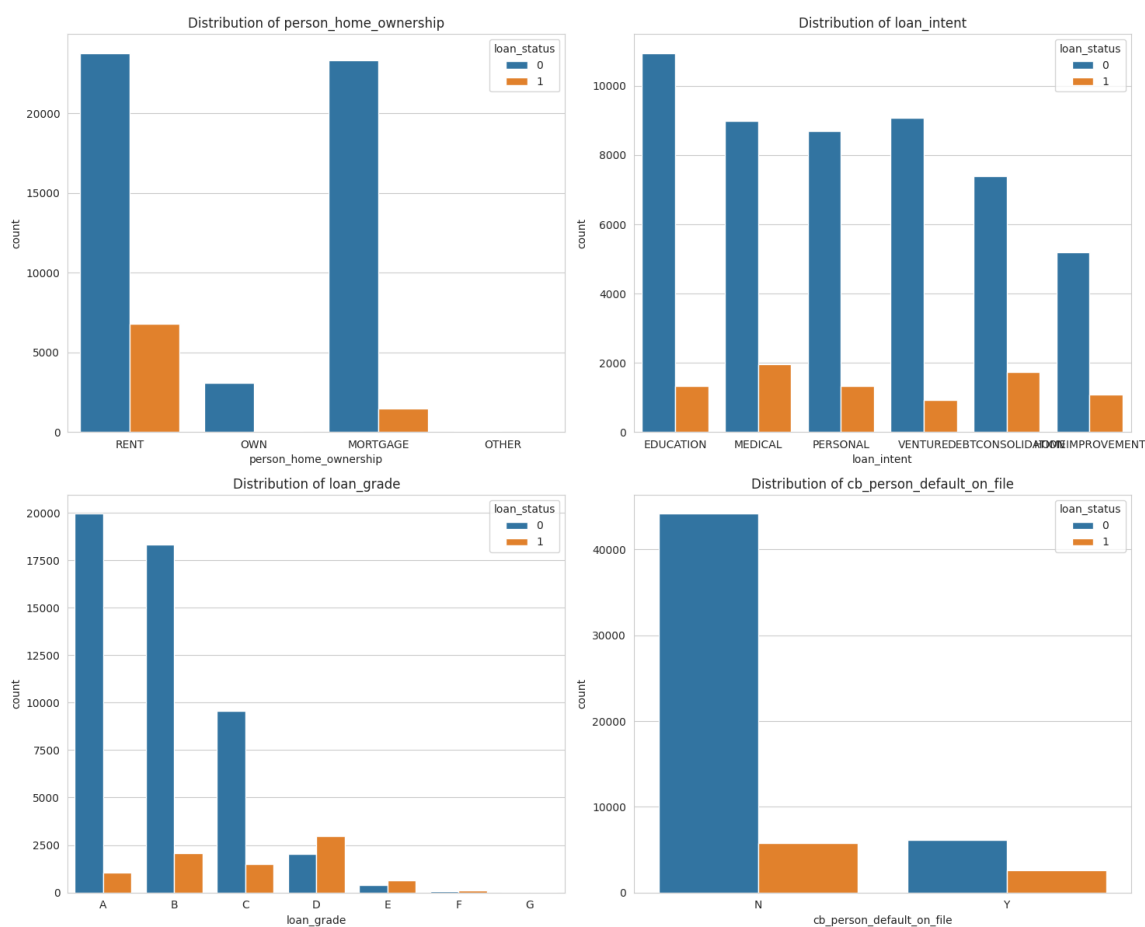
- **person_age:** Đa phần người vay có độ tuổi từ 20 đến 40, với phân phối lệch phải rõ rệt. Đáng chú ý, hồ sơ được duyệt ($\text{loan_status} = 1$) tập trung nhiều ở nhóm người trẻ hơn, cho thấy độ tuổi có thể là yếu tố ảnh hưởng đến khả năng được phê duyệt.
- **person_income:** Phân phối cũng bị lệch phải rất mạnh, với đa số người vay có thu nhập dưới 200,000. Các hồ sơ bị từ chối ($\text{loan_status} = 0$) có xu hướng xuất hiện ở cả mức thu nhập thấp và cao, trong khi hồ sơ được duyệt chủ yếu tập trung ở nhóm thu nhập thấp đến trung bình. Điều này cho thấy việc có thu nhập cao không đảm bảo khả năng được vay.
- **person_emp_length:** Độ dài thời gian làm việc hầu hết nằm dưới 20 năm. Tuy nhiên, loan_status không có xu hướng rõ ràng theo biến này, cho thấy ảnh hưởng của đặc trưng này đến kết quả phê duyệt là không mạnh.
- **loan_amnt:** Phân phối loan_amnt cho thấy phần lớn khoản vay có giá trị dưới 15,000. Hồ sơ được duyệt chiếm ưu thế ở mức khoản vay thấp, trong khi các khoản vay cao hơn thường bị từ chối. Điều này phản ánh chính sách thận trọng trong phê duyệt những khoản vay lớn.
- **loan_int_rate:** Lãi suất khoản vay phân bố đa đỉnh, dao động chủ yếu từ 5 đến 15%. Hồ sơ được duyệt có xu hướng tập trung ở mức lãi suất thấp đến trung bình. Điều này cũng phản ánh thực tế rằng lãi suất cao thường đi kèm với rủi ro lớn hơn.



- **loan_percent_income:** Đây là tỷ lệ giữa số tiền vay và thu nhập cá nhân. Phân phối cho thấy đa số người vay sử dụng dưới 40% thu nhập để chi trả khoản vay. Tỷ lệ được duyệt cao hơn rõ rệt ở nhóm có tỷ lệ này thấp, chứng tỏ đây là yếu tố quan trọng trong việc đánh giá khả năng trả nợ.
- **cb_person_cred_hist_length:** Đa số người vay có lịch sử tín dụng dưới 15 năm. Những người có lịch sử tín dụng dài hơn có vẻ được duyệt vay dễ dàng hơn. Điều này cho thấy độ dài lịch sử tín dụng là yếu tố có thể tăng độ tin cậy của khách hàng đối với tổ chức cho vay.

Qua các biểu đồ và phân tích trên, ta có thể thấy rằng một số đặc trưng số như `loan_amnt`, `loan_percent_income`, và `cb_person_cred_hist_length` có ảnh hưởng đáng kể đến xác suất được duyệt vay, và do đó có tiềm năng lớn trong việc trở thành đặc trưng đầu vào cho mô hình học máy.

Phân phối của các Đặc trưng phân loại



Hình 2.3: Phân phối các đặc trưng phân loại theo trạng thái khoản vay

Phân tích các đặc trưng phân loại giúp nhóm hiểu được xu hướng duyệt khoản vay theo từng nhóm thuộc tính rời rạc. Các đặc trưng được xem xét bao gồm: `person_home_ownership`, `loan_intent`, `loan_grade`, và `cb_person_default_on_file`.

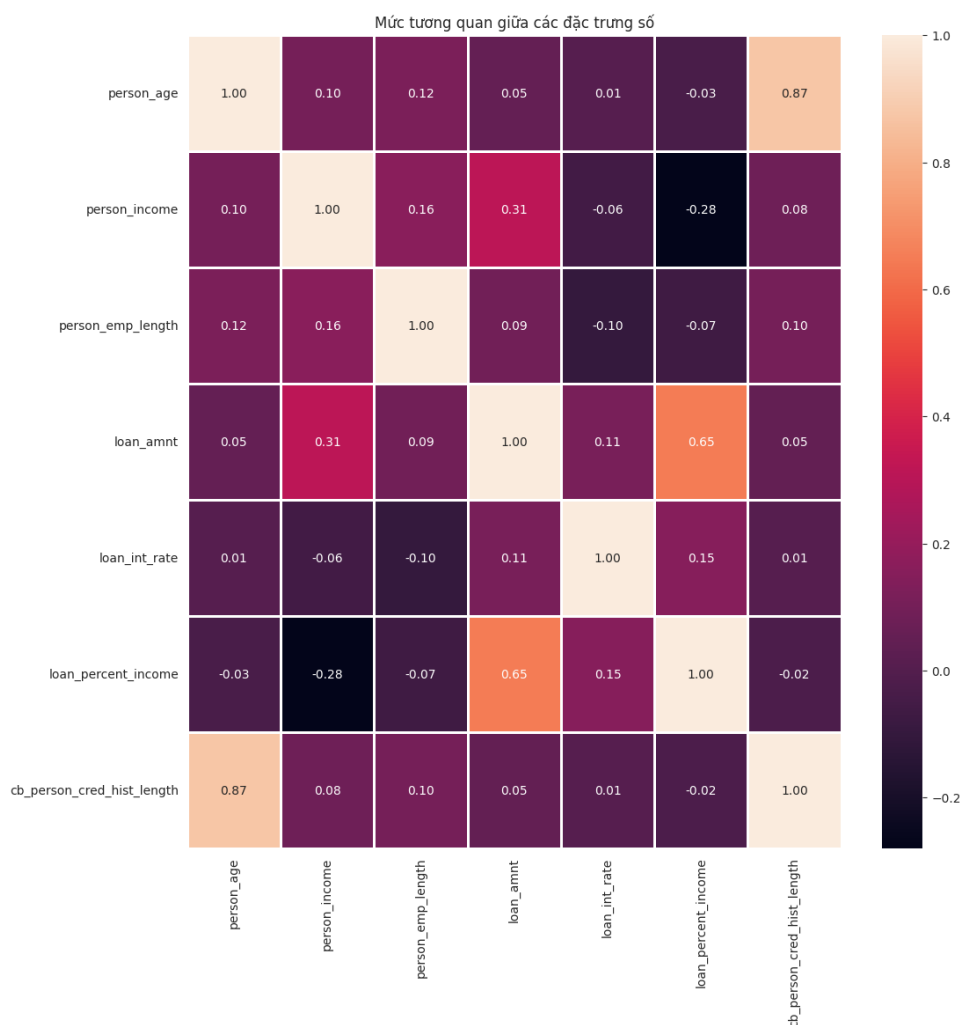


- **person_home_ownership:** Đa phần người vay thuộc nhóm RENT và MORTGAGE. Tuy nhiên, nhóm RENT có tỷ lệ hồ sơ được duyệt cao hơn so với các nhóm khác, trong khi nhóm OWN có tỷ lệ phê duyệt thấp nhất. Nhóm OTHER có số lượng mẫu rất ít nên không thể đưa ra kết luận chắc chắn. Biến này có thể phản ánh mức độ ổn định tài chính của người vay. Ngoài ra hình thức MORTGAGE là một hình thức thế chấp được hình thành từ rất sớm, từ thế kỷ 12-13 ở Trung cổ, phương Tây và hiện nay vẫn còn phổ biến hơn so với châu Á. Tài sản thế chấp có giá trị thấp, pháp lý không rõ ràng dẫn đến trả nợ hàng tháng cao hơn so với thu nhập, số lượng hồ sơ bị từ chối cũng gia tăng đáng kể.
- **loan_intent:** Các mục đích vay phổ biến nhất là DEBTCONSOLIDATION, PERSONAL, MEDICAL, và EDUCATION. Một số nhóm như VENTURE và HOMEIMPROVEMENT có ít dữ liệu hơn. Trong đó, các khoản vay với mục đích y tế (MEDICAL) và hợp nhất nợ (DEBTCONSOLIDATION) (nghĩa là thay vì có nhiều khoản nợ cùng các mức lãi suất vay khác nhau thì được hỗ trợ gộp về một khoản với lãi suất nhỏ hơn) có tỷ lệ được duyệt cao hơn, phản ánh sự ưu tiên từ phía tổ chức tài chính đối với các nhu cầu thiết yếu hoặc rủi ro thấp hơn.
- **loan_grade:** Số lượng khoản vay tập trung chủ yếu ở các hạng A, B và C. Có một xu hướng rõ rệt: tỷ lệ được duyệt tăng dần từ hạng A đến G, tức là những khoản vay có rủi ro cao hơn lại được duyệt nhiều hơn. Điều này có thể phản ánh chiến lược mở rộng cho vay đối với các đối tượng rủi ro cao khi nhu cầu tín dụng tăng mạnh. Bắt nguồn từ sau đại dịch 2019, chính phủ Mỹ đã rót tiền vào các chuỗi ngân hàng và phát triển lĩnh vực công nghệ tài chính. Cũng là lúc các ngân hàng nhận biết được nhu cầu tiêu dùng của mọi nhóm khách hàng đều gia tăng cùng với lợi nhuận lớn rất cao chỉ cần 20% khách hàng với hạng D đến G trả được khoản nợ.
- **cb_person_default_on_file:** Phần lớn người vay không có lịch sử nợ xấu (N). Tuy nhiên, bất ngờ là tỷ lệ được duyệt ở nhóm có nợ xấu (Y) lại cao hơn. Điều này gợi ý rằng các tổ chức tín dụng có thể vẫn chấp nhận rủi ro với những khách hàng đã từng vỡ nợ nếu các điều kiện khác đủ tốt.

Nhìn chung, các đặc trưng phân loại như `loan_intent` và `loan_grade` thể hiện sự tương quan mạnh với kết quả phê duyệt vay, từ đó có thể cung cấp thông tin giá trị cho mô hình dự đoán.



2.2.3 Data Visualization: Bivariate Analysis

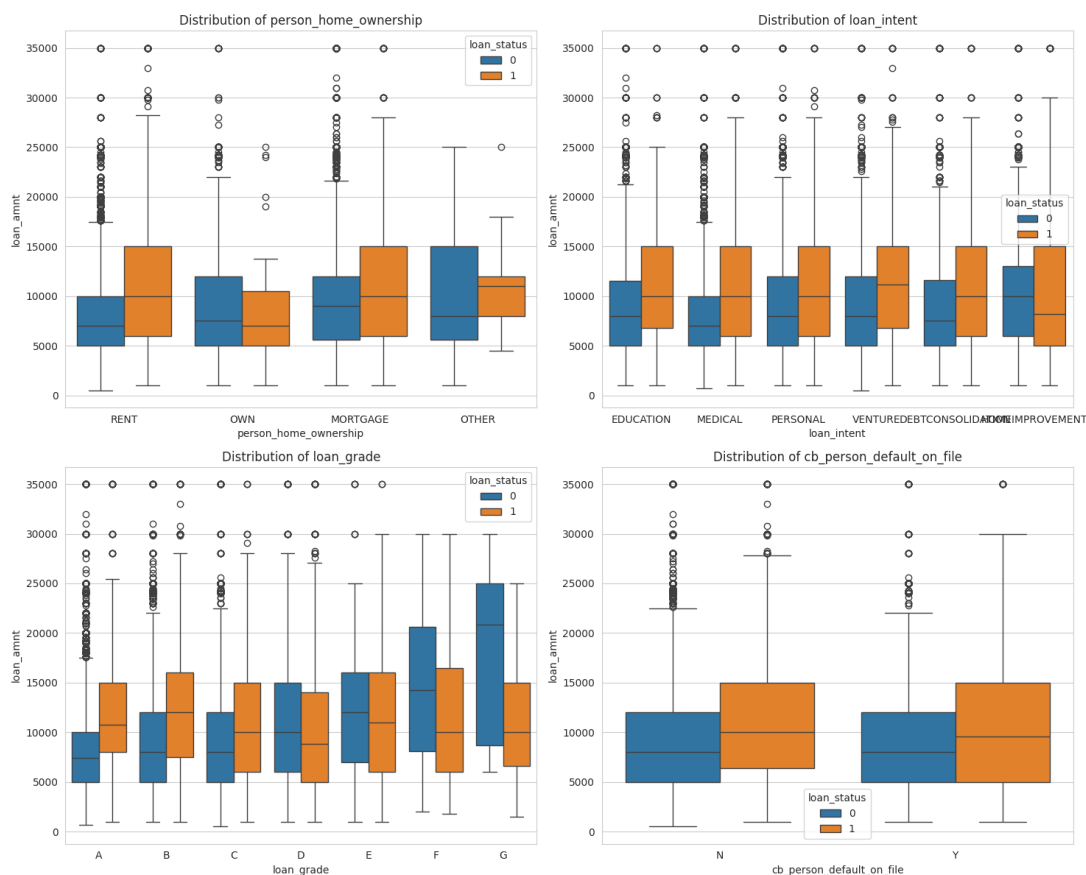


Hình 2.4: Ma trận tương quan giữa các đặc trưng số

- **Tuổi và lịch sử tín dụng:** Có mối tương quan rất mạnh (0.87) giữa `person_age` và `cb_person_cred_hist_length`, hợp lý vì người lớn tuổi thường có lịch sử tín dụng lâu hơn.
- **Số tiền vay và thu nhập:** Tương quan trung bình (0.31) giữa `loan_amnt` và `person_income`, cho thấy thu nhập cao thường vay được nhiều hơn.
- **Số tiền vay và tỷ lệ thu nhập trả nợ:** Mối tương quan mạnh (0.65) giữa `loan_amnt` và `loan_percent_income`, tức là khi khoản vay tăng, phần trăm thu nhập phải trả nợ cũng tăng.
- **Các mối tương quan còn lại:** Đa số có tương quan yếu (gần 0), điều này tốt cho mô hình vì các biến mang thông tin độc lập.



2.2.4 Phân tích phân phối theo các biến phân loại



Hình 2.5: Phân phối khoản vay theo các biến phân loại và trạng thái vay

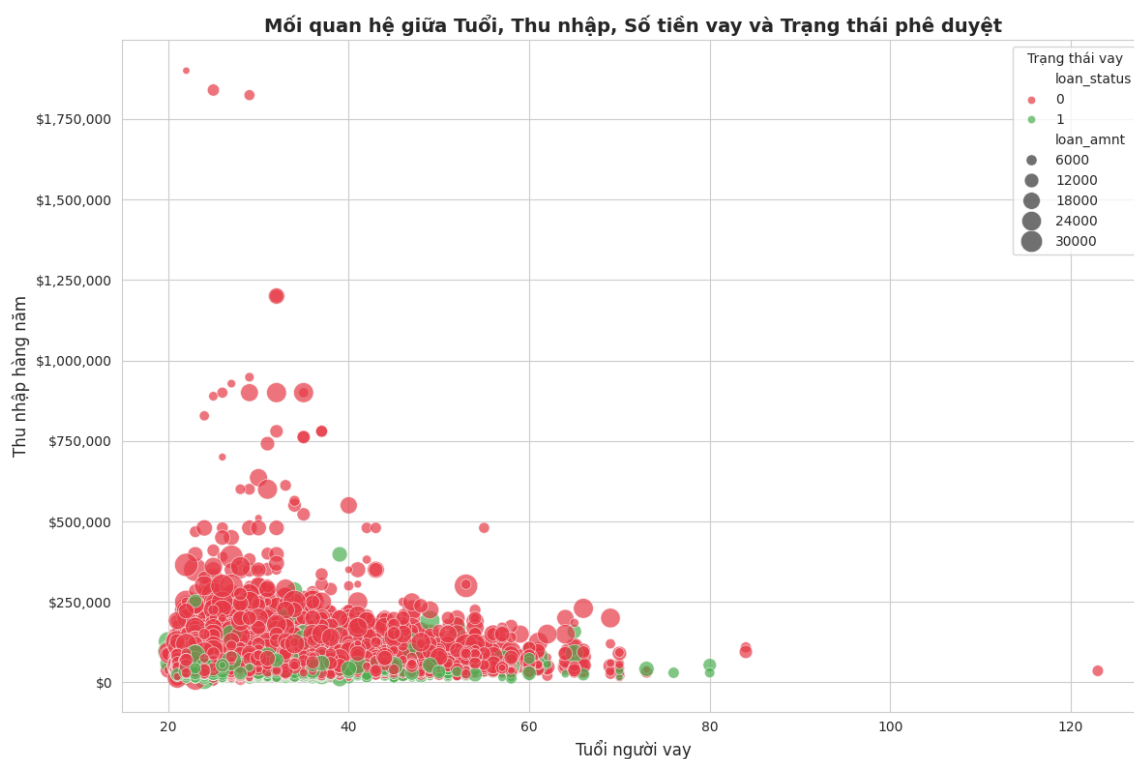
- **Tình trạng sở hữu nhà (person_home_ownership):** Người sở hữu nhà (OWN, MORTGAGE) thường vay nhiều hơn so với người thuê (RENT). Nhóm RENT cho thấy sự chênh lệch nhẹ về khoản vay giữa người được duyệt và bị từ chối.
- **Mục đích vay (loan_intent):** Các mục đích như DEBTCONSOLIDATION và HOMEIMPROVEMENT thường liên quan đến khoản vay lớn hơn. Trong nhóm VENTURE, người được duyệt có xu hướng vay nhiều hơn.
- **Chất lượng khoản vay (loan_grade):** Khoản vay trung bình tăng dần từ hạng A đến G, phản ánh mức độ rủi ro cao hơn thường đi kèm với khoản vay lớn hơn.
- **Lịch sử vỡ nợ (cb_person_default_on_file):** Người từng vỡ nợ (Y) có xu hướng vay nhiều hơn so với người chưa từng vỡ nợ (N), với độ biến động khoản vay cao hơn.

Các phân tích trên cho thấy một số đặc trưng như person_age, person_income, loan_percent_income, và cb_person_cred_hist_length có khả năng phân biệt tốt giữa các nhóm vay, do đó nên được giữ lại trong quá trình xây dựng mô hình.

Bên cạnh đó, các biến phân loại cũng thể hiện sự khác biệt rõ rệt giữa các trạng thái phê duyệt khoản vay, góp phần cung cấp thông tin giá trị cho việc dự báo rủi ro vỡ nợ.

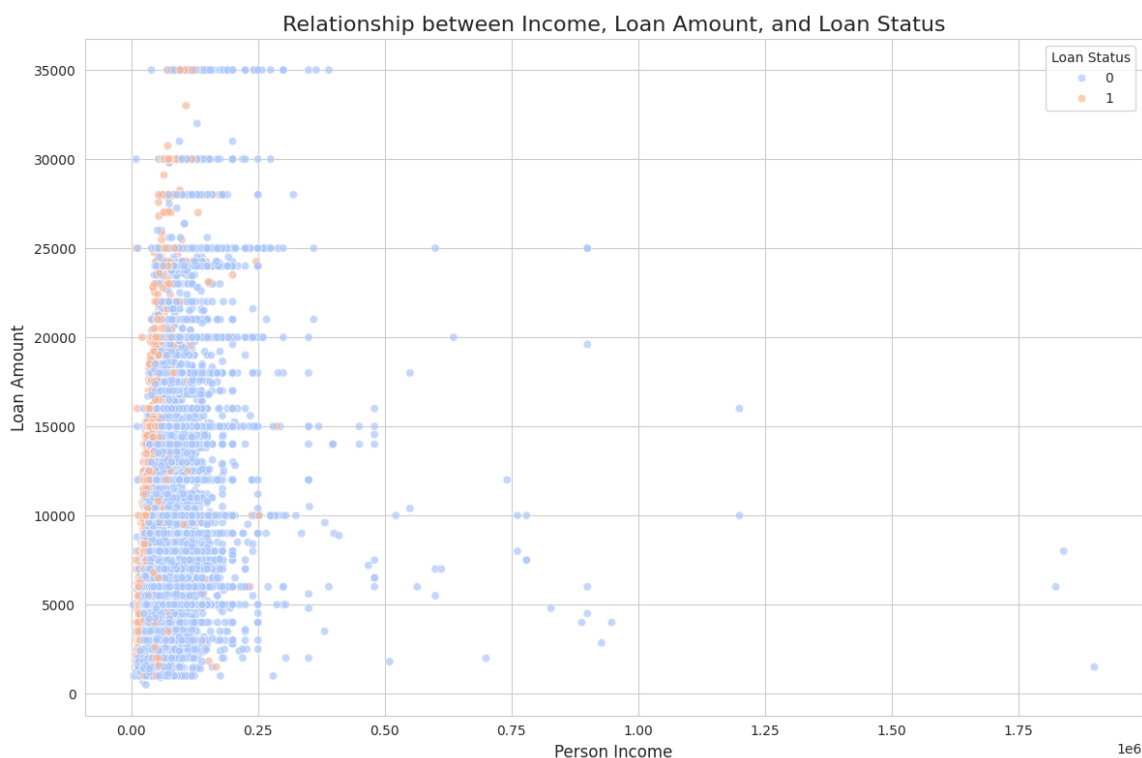


2.2.5 Data Visualization: Multivariate Analysis



Hình 2.6: Mối quan hệ giữa Tuổi, Thu nhập, Số tiền vay và Trạng thái phê duyệt

- **Kích thước điểm phản ánh số tiền vay:** Điểm lớn hơn biểu thị khoản vay lớn hơn. Các khoản vay lớn thường xuất hiện ở nhóm có thu nhập cao.
- **Tuổi và thu nhập:** Không thấy mối quan hệ tuyến tính rõ ràng. Người vay tập trung chủ yếu ở độ tuổi 20–40 với nhiều mức thu nhập khác nhau.
- **Thu nhập và trạng thái phê duyệt:** Tỷ lệ được duyệt (màu xanh lá cây) tăng ở mức thu nhập cao hơn. Người có thu nhập thấp thường bị từ chối.
- **Tổng hợp theo thu nhập:** Với thu nhập cao, cả khoản vay lớn và nhỏ đều có thể được duyệt; trong khi ở thu nhập thấp, các khoản vay lớn có xu hướng bị từ chối nhiều hơn.



Hình 2.7: Quan hệ giữa Thu nhập, Số tiền vay và Trạng thái vay

- **Trạng thái vay:** Màu xanh nhạt biểu thị hồ sơ bị từ chối (`loan_status = 0`), màu cam nhạt là được duyệt (`loan_status = 1`).
- **Thu nhập và số tiền vay:** Người có thu nhập cao thường vay số tiền lớn hơn. Ở mức thu nhập thấp, khoản vay thường nhỏ.
- **Thu nhập và trạng thái phê duyệt:** Ở mức thu nhập thấp, đa phần bị từ chối; thu nhập càng cao thì xác suất được duyệt càng tăng, mặc dù vẫn có ngoại lệ.
- **Tổng hợp:** Ở nhóm thu nhập thấp, cả khoản vay nhỏ và lớn đều khó được duyệt. Ở nhóm thu nhập cao, cả hai loại khoản vay có tỷ lệ duyệt cao hơn.

2.2.6 Kết luận cho giai đoạn EDA

Quá trình phân tích khám phá dữ liệu (EDA) đã mang lại những hiểu biết quan trọng:

- Xác định được các mối tương quan đáng chú ý, chẳng hạn như giữa `person_age` và `cb_person_cred_hist_length`, hay giữa `loan_amnt` và `person_income`.
- Các biến phân loại như `loan_intent`, `loan_grade` và `person_home_ownership` thể hiện sự phân bố khác biệt rõ ràng giữa các nhóm `loan_status`, cho thấy chúng có thể đóng vai trò quan trọng trong mô hình.
- Phát hiện hiện tượng mất cân bằng dữ liệu khi nhóm `loan_status = 1` chiếm tỷ trọng lớn hơn đáng kể so với nhóm còn lại.



- Xác định sự tồn tại của các giá trị ngoại lai (outliers) trong các biến số như thu nhập, độ tuổi và số tiền vay, đòi hỏi cần xử lý cẩn trọng trong giai đoạn tiền xử lý.
- Phân tích đa biến cho thấy rằng không chỉ thu nhập, mà sự kết hợp giữa nhiều yếu tố (tuổi, thu nhập, số tiền vay, mục đích vay...) ảnh hưởng đến khả năng phê duyệt khoản vay.

Những thông tin này sẽ là cơ sở quan trọng để đề xuất chiến lược tiền xử lý dữ liệu, lựa chọn đặc trưng và tối ưu mô hình dự đoán.

Chương 3

TỔNG QUAN HƯỚNG TIẾP CẬN VÀ PHƯƠNG PHÁP

3.1 Các mô hình học máy sử dụng

Khi làm việc với dữ liệu dạng bảng (tabular data), các mô hình học máy dựa trên cây quyết định thường mang lại hiệu quả tốt hơn các mô hình học sâu như MLP hoặc DNN. Điều này là bởi vì:

- Mô hình MLP thường tạo ra giải pháp “mịn màng” không phù hợp với đặc tính dữ liệu dạng bảng có cấu trúc rời rạc và phi tuyến.
- Mô hình dựa trên cây có khả năng tự động lựa chọn đặc trưng quan trọng mà không cần tiền xử lý nhiều.
- Chúng có khả năng mô hình hoá các quan hệ phi tuyến và có tính giải thích cao.

Từ những lý do trên, nhóm đã chọn ba mô hình tree-based nổi bật:

- **CatBoostClassifier** [Pro+19]: hỗ trợ tốt dữ liệu phân loại, tự xử lý missing values và tránh overfitting hiệu quả.
- **LGBMClassifier** [Ke+17]: có tốc độ huấn luyện nhanh và hiệu quả cao với dữ liệu lớn.
- **XGBClassifier** [CG16]: tối ưu hiệu suất, có khả năng chống overfitting và rất phổ biến trong các cuộc thi.

3.2 Metric đánh giá mô hình

Các metric được nhóm sử dụng để đánh giá hiệu quả mô hình bao gồm:

- **Accuracy**: tỷ lệ dự đoán đúng trên toàn bộ tập dữ liệu.
- **F1-score**: trung bình điều hòa giữa precision và recall, phù hợp với bài toán dữ liệu mất cân bằng.



- **AUC (Area Under the Curve):** Đo lường khả năng mô hình phân biệt giữa hai lớp. AUC cao cho thấy mô hình có khả năng phân biệt tốt giữa các đối tượng được và không được duyệt vay.

Nhóm tập trung tối ưu hoá Accuracy và F1-score vì đây là những chỉ số phù hợp nhất để đánh giá hiệu quả của mô hình phân loại với dữ liệu mất cân bằng giữa các lớp (được duyệt/không được duyệt vay).

3.3 Feature Engineering và xử lý dữ liệu

Feature engineering là một trong những bước quan trọng nhất trong pipeline, giúp cải thiện khả năng học của mô hình.

Các bước xử lý bao gồm:

- **Xử lý missing values:** Các cột thiếu dữ liệu được xử lý bằng phương pháp trung bình, mode hoặc loại bỏ tùy theo mức độ ảnh hưởng.
- **Xử lý outliers:** Áp dụng kỹ thuật IQR để loại bỏ ngoại lai tại các cột nhạy cảm như *person_age*, *loan_int_rate*, *loan_amnt*.
- **Tạo đặc trưng mới:** Ví dụ: $\text{loan_percent_income} = \text{loan_amnt} / \text{person_income}$.
- **Mã hóa dữ liệu phân loại:** Sử dụng OneHotEncoder cho các cột không thứ tự và OrdinalEncoder cho các cột có thứ tự như *loan_grade*.
- **Chuẩn hoá dữ liệu:** Sử dụng MinMaxScaler để đưa các đặc trưng số về cùng thang đo nhằm tránh ảnh hưởng bất thường lên mô hình.
- **Chọn đặc trưng:** Áp dụng Correlation Heatmap, feature importance từ mô hình, và phương pháp thống kê để loại bỏ các cột không liên quan.

3.4 Kết hợp mô hình và đánh giá

Sau khi huấn luyện riêng lẻ từng mô hình, nhóm sử dụng kỹ thuật Ensemble để kết hợp kết quả của ba mô hình CatBoost, LightGBM và XGBoost nhằm cải thiện độ chính xác và ổn định.

Cụ thể:

- **Voting Classifier:** Áp dụng soft voting từ kết quả của ba mô hình base để tạo ra dự đoán cuối cùng.
- **Cross Validation (K-Fold):** Sử dụng 5-fold để đánh giá mô hình một cách toàn diện, tránh overfitting.
- **Tối ưu hoá siêu tham số (Hyperparameter Tuning):** Áp dụng GridSearchCV để tìm tổ hợp tham số tốt nhất cho từng mô hình.

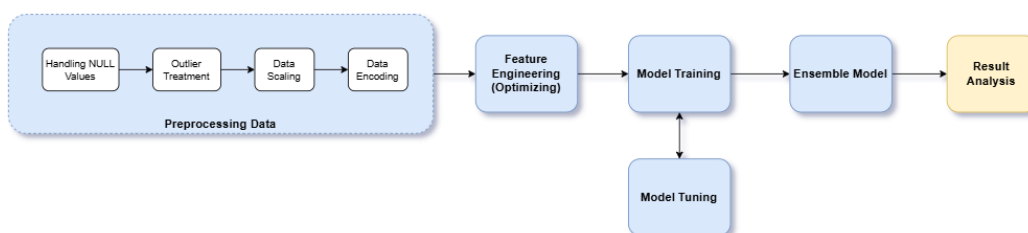
Qua đó, nhóm đảm bảo kết quả đánh giá không phụ thuộc vào sự chia tách dữ liệu và tối ưu hiệu suất dự đoán trong môi trường thực tế.

Chương 4

THỰC NGHIỆM

4.1 Pipeline

Dựa trên các bước đã thực hiện trong quá trình thực nghiệm, nhóm chúng tôi đã xây dựng một pipeline tổng thể để xử lý bài toán phân loại tín dụng, như minh họa trong Hình 4.1.



Hình 4.1: Pipeline tổng thể

Pipeline này gồm các giai đoạn chính sau:

1. Tiền xử lý dữ liệu (Preprocessing Data):

- *Handling NULL Values*: Loại bỏ hoặc thay thế các giá trị thiếu bằng các chiến lược phù hợp (mean, median hoặc constant).
- *Outlier Treatment*: Phát hiện và xử lý các giá trị ngoại lai thông qua các biểu đồ phân phối hoặc phương pháp thống kê như IQR.
- *Data Scaling*: Chuẩn hóa dữ liệu nhằm đưa các đặc trưng về cùng một thang đo (ví dụ: MinMaxScaler, StandardScaler).
- *Data Encoding*: Mã hóa các biến phân loại bằng Label Encoding hoặc One-Hot Encoding để mô hình có thể xử lý.

2. Feature Engineering (Optimizing):

- Tạo ra các đặc trưng mới dựa trên domain knowledge.



- Loại các đặc trưng không quan trọng hoặc gây nhiễu để nâng cao độ chính xác của mô hình.

3. Huấn luyện mô hình (Model Training):

- Sử dụng các thuật toán mạnh như *CatBoostClassifier*, *LGBMClassifier* và *XGBClassifier*.

4. Tối ưu hóa mô hình (Model Tuning):

- Tinh chỉnh siêu tham số bằng *GridSearchCV* để cải thiện hiệu suất.

5. Tổng hợp mô hình (Ensemble Model):

- Kết hợp các mô hình đã huấn luyện bằng hai phương pháp Voting và Stacking nhằm tận dụng ưu điểm của từng mô hình riêng lẻ.

6. Phân tích kết quả (Result Analysis):

- Đánh giá mô hình bằng các chỉ số như Accuracy, F1-score và rút ra kết luận.

Pipeline được thiết kế để dễ dàng mở rộng, cải tiến và tái sử dụng trong các bài toán tương tự.

4.2 Tiền xử lý dữ liệu

4.2.1 Các chiến thuật xử lý NULL

Để chắc chắn, ta cần kiểm tra xem Tập kiểm tra (test dataset) có giá trị NULL hay không để có thể áp dụng các chiến lược nhất quán (nếu có) với train dataset

	0
id	0
person_age	0
person_income	0
person_home_ownership	0
person_emp_length	0
loan_intent	0
loan_grade	0
loan_amnt	0
loan_int_rate	0
loan_percent_income	0
cb_person_default_on_file	0
cb_person_cred_hist_length	0
loan_status	0

Bảng 4.1: kết quả của dòng lệnh `df_train.isna().sum()` và `df_test.isna().sum()`

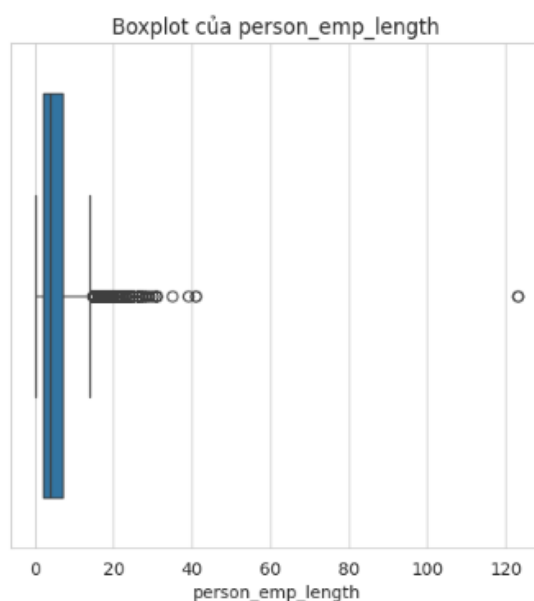
Ta nhận thấy rằng DataFrame này (cả train dataset và test dataset) không bị khuyết dữ liệu. Vì vậy, ở bước này, chúng ta không cần áp dụng bất kỳ chiến thuật xử lý NULL nào cho tập huấn luyện.



4.2.2 Các chiến thuật xử lý outlier

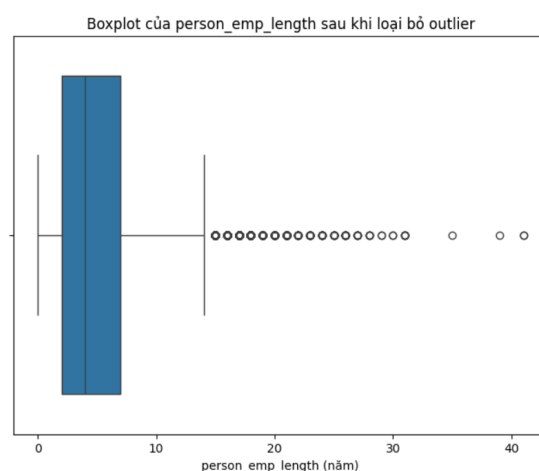
Ở bước này ta cần vẽ các biểu đồ giúp trực quan hóa sự phân tán của dữ liệu và xác định các giá trị ngoại lai (outliers) một cách rõ ràng hơn

Xử lý outliers ở cột *person_emp_length*



Hình 4.2: Biểu đồ boxplot của *person_emp_length*

Ở biểu đồ về số năm kinh nghiệm làm việc của ứng viên, hộp tập trung ở thời gian làm việc ngắn, dưới khoảng 10 năm. Chúng ta thấy rõ giá trị outlier 123 mang tính bất thường và phi thực tế trong bối cảnh thông thường. Tuổi thọ trung bình của con người hiện nay hiếm khi và hầu như là không vượt quá con số này, vì vậy không thể tồn tại số năm làm việc liên tục lên đến con số này. Ngoài ra, cũng có một số ít đáng ngờ ở phía trên (trên 40-50 năm).

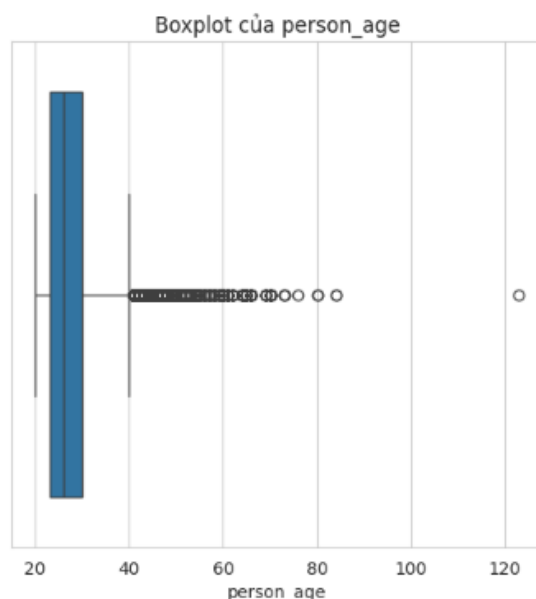


Hình 4.3: Biểu đồ boxplot của *person_emp_length* sau khi xử lý

Kích thước DataFrame trước và sau khi loại bỏ outlier lần lượt: (58645, 13) và (58643, 13)



Xử lý các outliers ở cột *person_age*



Hình 4.4: Biểu đồ boxplot của *person_age*

Các box tập trung chủ yếu ở độ tuổi khoảng 20 đến 35. Đường trung vị (median) nằm trong khoảng 25-30 tuổi. Để ý thấy có rất nhiều outliers lại ở phía trên, đặc biệt là các giá trị trên 60 tuổi, và một giá trị rất cao (123). Sau khi xem xét về nguồn gốc dữ liệu thì ta thấy lượng cho vay thế chấp rất cao nên dữ liệu được lấy từ các đối tượng tại phương tây. Ở đây tuổi 60 - 80 là điều bình thường và 123 là một độ tuổi phi lý cần được loại bỏ.

Thay vì áp dụng các phương pháp thống kê thuần túy để xác định và loại bỏ outliers dựa trên khoảng tứ phân vị (IQR) hoặc độ lệch chuẩn, chúng ta kết hợp xem xét các yếu tố sau:

- **Ngưỡng pháp lý:** Theo quy định pháp luật về khả năng vay vốn, độ tuổi tối thiểu thường là 18 tuổi. Do đó, các trường hợp có *person_age* dưới 18 được coi là không hợp lệ và đã được loại bỏ khỏi tập dữ liệu.
- **Phân phối dữ liệu và tính hợp lệ:** Quan sát từ biểu đồ boxplot cho thấy phần lớn dữ liệu tập trung trong khoảng 20 - 60 tuổi. Mặc dù có các giá trị nằm ngoài phạm vi này, chúng tôi nhận thấy các giá trị trong khoảng 65 - 80 tuổi vẫn có thể đại diện cho các cá nhân hợp lệ có nhu cầu vay vốn. Việc loại bỏ các trường hợp này có thể dẫn đến mất mát thông tin hữu ích cho mô hình.

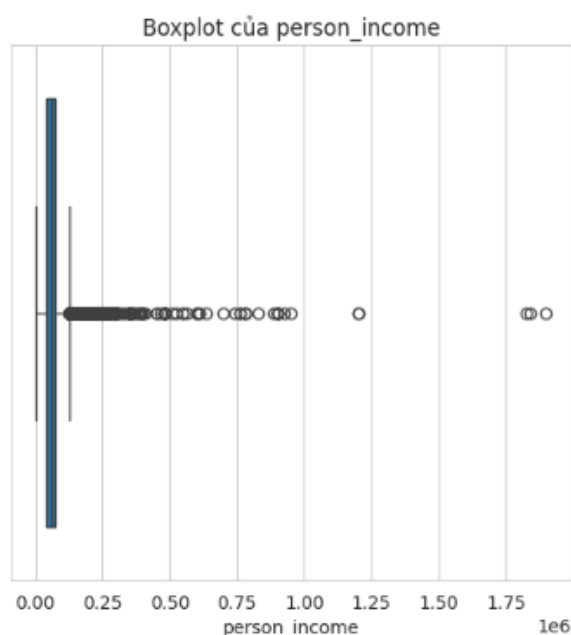
Dựa trên những phân tích trên, chúng ta sẽ thực hiện các bước sau:

- Loại bỏ các bản ghi có *person_age* nhỏ hơn 18. Điều này nhằm đảm bảo tính tuân thủ pháp lý của dữ liệu và loại bỏ các trường hợp không đủ điều kiện vay vốn.
- Giữ nguyên các bản ghi có *person_age* trong khoảng từ 18 đến 80. Chúng tôi cho rằng các giá trị này vẫn nằm trong phạm vi có thể chấp nhận được của độ tuổi người vay và có thể chứa thông tin quan trọng cho việc huấn luyện mô hình.

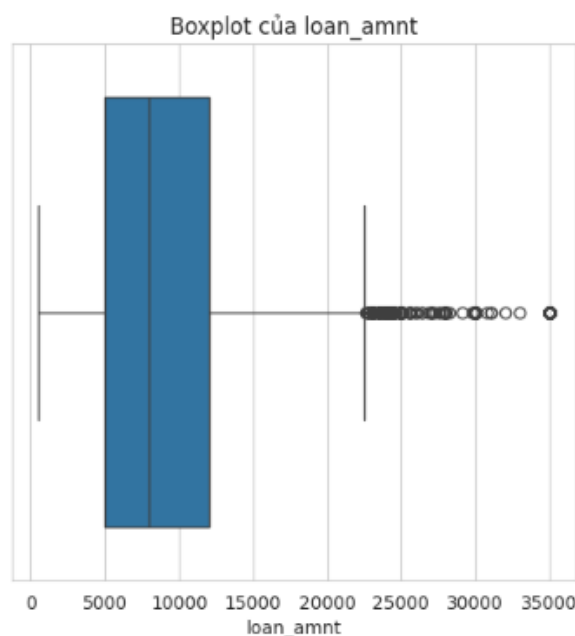
Kích thước DataFrame trước và sau khi loại bỏ outlier lần lượt: (58643, 13) và (58640, 13)



Xử lý ở hai cột *person_income* và *loan_amnt*



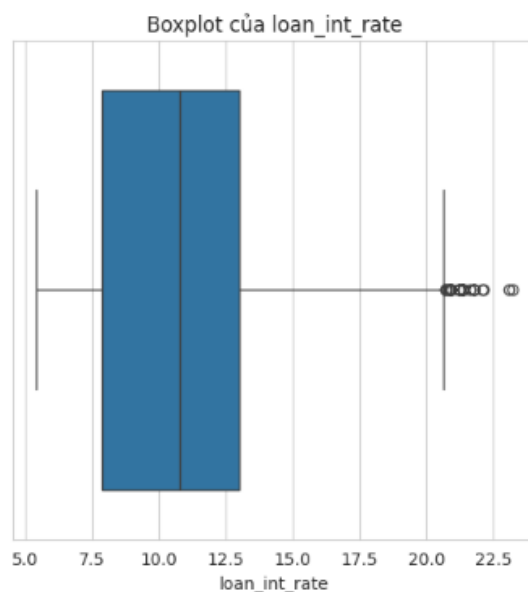
Hình 4.5: Biểu đồ boxplot của *person_income*



Hình 4.6: Biểu đồ boxplot của *loan_amnt*

Đề xuất phương pháp: Giữ nguyên các giá trị hiện tại. Lý do: tính thực tế (có thể chứa các thông tin quan trọng)

- **Đối với *person_income*:** Thu nhập cao là một yếu tố quan trọng và không nhất thiết là bất thường. Mô hình có thể học được mối quan hệ giữa thu nhập cao và khả năng trả nợ.
- **Đối với *loan_amnt*:** Số tiền vay lớn là một phần tự nhiên của nghiệp vụ cho vay, đặc biệt đối với những người có thu nhập cao.

**Xử lý outliers ở cột *loan_int_rate***

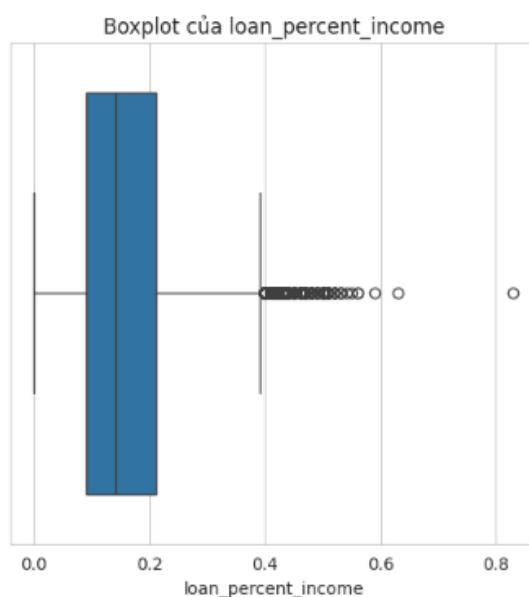
Hình 4.7: Biểu đồ boxplot của loan_int_rate

Dải giá trị tập trung từ 6% - 15% và có nhiều outlier vượt 20% trở lên cao bất thường so với phần lớn mẫu. Lãi suất quá cao thường chỉ ra khách hàng có rủi ro cao hoặc lỗi nhập liệu. Vì vậy sử dụng phương pháp điền khuyết bằng median.

Lý do lựa chọn phương pháp điền khuyết bằng giá trị trung vị (median) cho các giá trị ngoại lai cao:

- **Tính Robust:** Giá trị trung vị ít bị ảnh hưởng bởi các giá trị cực đoan so với giá trị trung bình.
- **Bảo toàn phân phối chung:** Việc điền khuyết bằng median của các giá trị "bình thường" giúp duy trì hình dạng chung của phân phối lãi suất.

Lưu ý: Ngưỡng dưới: cần xem xét đến các giá trị lãi suất quá thấp (dưới 5%) là outliers không? Trong nghiệp vụ cho vay, lãi suất thường có một mức sàn nhất định. Tuy nhiên, với dữ liệu này, có vẻ như các giá trị thấp không quá nhiều và có thể là các chương trình ưu đãi đặc biệt. Quyết định không xử lý chúng ở bước này có thể chấp nhận được.

**Xử lý outliers ở cột *loan_percent_income***Hình 4.8: Biểu đồ boxplot của *loan_percent_income*

Quan sát từ biểu đồ boxplot cho thấy phần lớn các giá trị của biến *loan_percent_income* nằm dưới mức 0.4. Tuy nhiên, xuất hiện một số ngoại lệ (outliers) có giá trị dao động từ 0.6 đến 0.8. Những giá trị này được xem là bất thường và có khả năng không phản ánh đúng thực tế.

Theo dữ liệu từ Cục Dự trữ Liên bang Hoa Kỳ (Federal Reserve), tỷ lệ chi trả nợ trên thu nhập khả dụng (debt service ratio) trung bình của các hộ gia đình Mỹ vào năm 2023 là khoảng 9.8%. Điều này cho thấy chỉ khoảng 10% thu nhập của hộ gia đình được sử dụng để thanh toán các khoản nợ định kỳ như thế chấp, vay tiêu dùng và các khoản tín dụng khác.

Để minh họa, giả sử một cá nhân có tổng thu nhập hàng năm là 45,000 USD, tương đương 3,750 USD mỗi tháng. Nếu chi phí nhà ở hàng tháng là 1,050 USD, tỷ lệ chi phí nhà ở so với thu nhập là 28% ($1,050/3,750$). Trường hợp tổng nghĩa vụ thanh toán nợ hàng tháng, bao gồm cả tiền nhà, là 1,350 USD, thì tỷ lệ hoàn vốn sẽ là 36% ($1,350/3,750$).

Như vậy, các tỷ lệ từ 0.6 đến 0.8 (tức 60%–80% thu nhập dùng để trả nợ) là rất cao và vượt quá mức thông thường. Một số tài liệu chỉ ra rằng ngưỡng tối đa có thể chấp nhận được trong các tình huống đặc biệt chỉ vào khoảng 45%. Do đó, các giá trị trên 0.6 có thể được xem là outlier và cần loại bỏ trong bước tiền xử lý dữ liệu.

Kích thước tập dữ liệu trước và sau khi loại bỏ outlier lần lượt là: **(58640, 13)** và **(58601, 13)**.

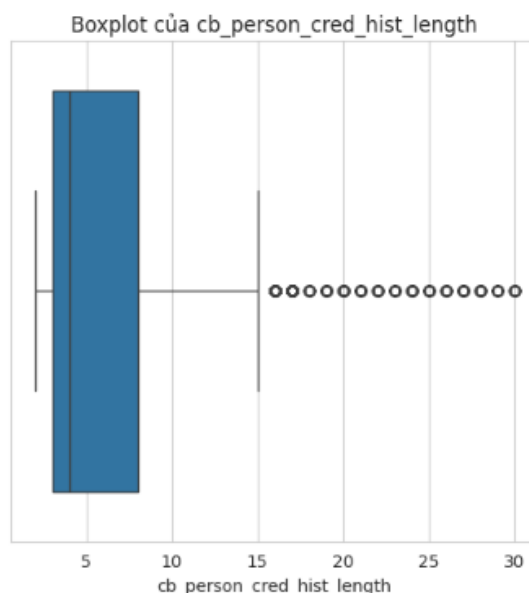
Nhận xét về việc xử lý outliers ở *loan_percent_income*:

- **Mức độ loại bỏ chấp nhận được:** Việc loại bỏ 39 bản ghi trong tổng số hơn 58,000 (chưa đến 1%) được xem là hợp lý. Mức mất mát dữ liệu là nhỏ và hoàn toàn có thể chấp nhận, đặc biệt khi các bản ghi bị loại bỏ là ngoại lệ rõ ràng.
- **Căn cứ vững chắc:** Quyết định loại bỏ các giá trị lớn hơn 0.6 dựa trên các thống kê và



phân tích thực tế về khả năng chi trả của người vay, giúp mô hình học máy tránh được nhiễu từ dữ liệu không đại diện.

Xử lý ở cột *cb_person_cred_hist_length*



Hình 4.9: Biểu đồ boxplot của *cb_person_cred_hist_length*

Thời gian có lịch sử tín dụng thường từ vài năm đến 10–15 năm. Trong ngành ngân hàng, thông tin lịch sử tín dụng có giá trị rất lớn. Nếu phát hiện giá trị không hợp lý (quá cao), thường cần kiểm tra lại với bộ phận dữ liệu thay vì loại bỏ hoàn toàn. Việc điền khuyết bằng median giúp giảm nhiễu mà không mất thông tin quan trọng.

Nhận xét: Phương pháp điền khuyết bằng giá trị trung vị (median) cho các giá trị không hợp lý (ngoài phạm vi 0-30 năm) là một lựa chọn ổn vì:

- **Bảo toàn thông tin:** Thay vì loại bỏ hoàn toàn, chúng ta giữ lại bản ghi và chỉ điều chỉnh giá trị của cột có vấn đề.
- **Giảm nhiễu:** Việc điền khuyết bằng median giúp giảm tác động của các giá trị ngoại lai mà không làm thay đổi đáng kể phân phối chung của cột.
- **Phù hợp với nghiệp vụ:** Nhận xét về việc ngân hàng thường không loại bỏ hoàn toàn outliers trong các cột quan trọng như lịch sử tín dụng là rất chính xác.

4.2.3 Mã hoá Dữ liệu (Encoding)

Trước tiên, cột *id* được loại bỏ khỏi tập dữ liệu vì đây chỉ là chỉ số định danh, không mang thông tin có giá trị trong việc dự đoán khả năng vay vốn.

Dựa trên phân tích khám phá dữ liệu (EDA), các biến phân loại (categorical) cần được mã hoá bao gồm:



- **person_home_ownership:** ['RENT', 'OWN', 'MORTGAGE', 'OTHER'] (Nominal)
- **loan_intent:** ['EDUCATION', 'MEDICAL', 'PERSONAL', 'VENTURE', 'DEBTCONSOLIDATION', 'HOMEIMPROVEMENT'] (Nominal)
- **loan_grade:** ['B', 'C', 'A', 'D', 'E', 'F', 'G'] (Ordinal)
- **cb_person_default_on_file:** ['N', 'Y'] (Binary)

Phương pháp mã hoá được áp dụng:

- **One-Hot Encoding:** Áp dụng cho các biến dạng danh mục không thứ tự (nominal), bao gồm `person_home_ownership` và `loan_intent`. Sử dụng hàm `pd.get_dummies()` để tạo các cột nhị phân tương ứng cho từng giá trị duy nhất trong các biến này. Tham số `prefix` được sử dụng nhằm đặt tiền tố cho tên các cột mới giúp dễ nhận diện, và `drop_first=True` được thiết lập để tránh hiện tượng đa cộng tuyến (multicollinearity) bằng cách loại bỏ một cột đại diện cho mỗi biến.
- **Label Encoding:** Được áp dụng cho các biến có thứ tự hoặc nhị phân, bao gồm `loan_grade` và `cb_person_default_on_file`. Với `loan_grade`, việc mã hoá tuân theo thứ tự $A > B > C > D > E > F > G$ nhằm bảo toàn ý nghĩa xếp hạng. Đối với `cb_person_default_on_file`, các giá trị được ánh xạ rõ ràng thành: 'N' $\rightarrow$ 0 và 'Y' $\rightarrow$ 1. Quá trình mã hoá được thực hiện bằng `LabelEncoder` từ thư viện `sklearn.preprocessing`, và các cột gốc sẽ được loại bỏ sau khi mã hoá.

Sau bước này, `DataFrame df_train` sẽ chỉ còn các biến định lượng (numeric), trong đó các biến phân loại đã được chuyển đổi thành dạng số, sẵn sàng cho bước chuẩn hoá dữ liệu hoặc đưa trực tiếp vào mô hình học máy.



#	Tên Cột	Số lượng không rỗng	Kiểu Dữ Liệu
0	person_age	58601	int64
1	person_income	58601	int64
2	person_emp_length	58601	float64
3	loan_amnt	58601	int64
4	loan_int_rate	58601	float64
5	loan_percent_income	58601	float64
6	cb_person_cred_hist_length	58601	float64
7	loan_status	58601	int64
8	home_ownership_OTHER	58601	bool
9	home_ownership_OWN	58601	bool
10	home_ownership_RENT	58601	bool
11	loan_intent_EDUCATION	58601	bool
12	loan_intent_HOMEIMPROVEMENT	58601	bool
13	loan_intent_MEDICAL	58601	bool
14	loan_intent_PERSONAL	58601	bool
15	loan_intent_VENTURE	58601	bool
16	loan_grade_encoded	58601	int64
17	cb_person_default_on_file_encoded	58601	int64

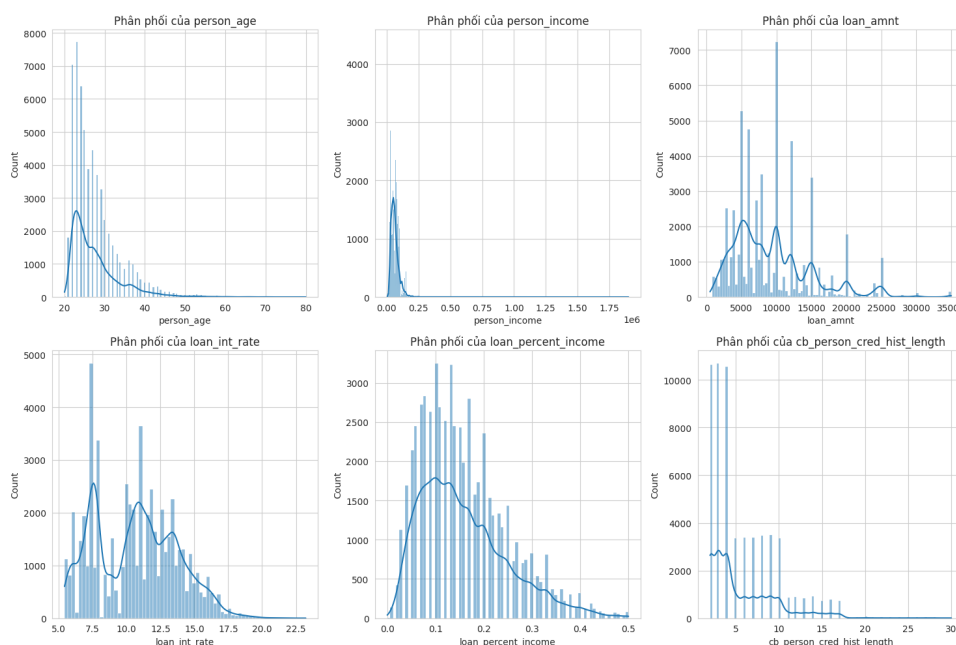
Bảng 4.2: Thông tin tổng quan về các cột dữ liệu sau khi mã hoá

4.2.4 Chuẩn hoá dữ liệu - Data Scaling

Xem lại dữ liệu trước khi chọn phương pháp chuẩn hoá

Chuẩn hóa dữ liệu là một giai đoạn tiền xử lý quan trọng, áp dụng các kỹ thuật nhằm biến đổi giá trị của các cột số trong tập dữ liệu *df_train* về một thang đo chung. Mục tiêu của quá trình này là bảo toàn sự khác biệt tương đối giữa các phạm vi giá trị, đồng thời tối ưu hóa hiệu suất của mô hình học máy bằng cách cải thiện tốc độ hội tụ và tiềm năng nâng cao độ chính xác.

Tuy nhiên, việc áp dụng chuẩn hóa dữ liệu cần được xem xét dựa trên đặc điểm phân phối của từng thuộc tính và yêu cầu của thuật toán học máy được lựa chọn. Không phải mọi tập dữ liệu đều hưởng lợi từ quá trình chuẩn hóa, và việc áp dụng không cần thiết có thể không mang lại cải thiện đáng kể.



Hình 4.10: Biểu đồ phân phối của các đặc trưng

Dựa trên quan sát từ các biểu đồ phân phối và giá trị cực đại (max value), có thể đưa ra một số nhận định về đặc điểm phân phối của các biến liên tục như sau:

- **person_age** (Giá trị tối đa: 80): Phân phối có xu hướng **lệch phải nhẹ**, với phần lớn dữ liệu tập trung ở nhóm tuổi trẻ (20–40). Đường KDE cho thấy một đỉnh chính và đuôi mỏng kéo dài về phía độ tuổi cao hơn.
- **person_income** (Giá trị tối đa: 1,900,000): Phân phối thể hiện sự **lệch phải nghiêm trọng**. Đa số quan sát tập trung ở mức thu nhập thấp, trong khi một số ít cá nhân có thu nhập rất cao tạo thành một đuôi dài bên phải. Đường KDE có đỉnh rất cao ở gần 0 và giảm dần rất chậm.
- **loan_amnt** (Giá trị tối đa: 35,000): Phân phối có xu hướng **lệch phải**, mặc dù có thể có một vài đỉnh nhỏ. Phần lớn các khoản vay có giá trị thấp, với một số khoản vay lớn tạo thành đuôi bên phải.
- **loan_int_rate** (Giá trị tối đa: 23.22%): Phân phối **gần giống phân phối chuẩn**, với một đỉnh rõ ràng trong khoảng 5–15%. Tuy nhiên, vẫn có một chút lệch phải do một số khoản vay có lãi suất cao hơn mức trung bình.
- **loan_percent_income** (Giá trị tối đa: 0.5): Sau khi xử lý các outlier, phân phối vẫn **lệch phải rõ rệt**, với phần lớn các giá trị nằm dưới mức 0.2. Đuôi kéo dài đến ngưỡng 0.5, nhưng số lượng giá trị cao là rất ít.
- **cb_person_cred_hist_length** (Giá trị tối đa: 30): Phân phối thể hiện **lệch phải nghiêm trọng**, với phần lớn giá trị tập trung dưới 10 năm. Đuôi dài kéo về phía 30 năm cho thấy một số ít cá nhân có lịch sử tín dụng dài hơn nhiều so với phần còn lại.



Tại sao biểu đồ phân phối lại quan trọng trong việc chọn scaler?

- **MinMaxScaler (Normalization):** Thường được ưu tiên khi dữ liệu có một phạm vi hữu hạn và không có quá nhiều outliers. Nó co giãn dữ liệu về một phạm vi cố định (thường $[0, 1]$). Nếu phân phối không phải là Gaussian, MinMaxScaler có thể giữ nguyên hình dạng phân phối ban đầu tốt hơn StandardScaler.
- **StandardScaler (Z-score scaling):** Chuẩn hóa dữ liệu để có mean là 0 và standard deviation là 1. Nó hữu ích khi dữ liệu có phân phối gần Gaussian hoặc khi các thuật toán (ví dụ: SVM, linear regression) giả định dữ liệu có phân phối chuẩn. Nó ít bị ảnh hưởng bởi outliers hơn MinMaxScaler.
- **Phân phối lệch:** Nếu một cột có phân phối lệch (ví dụ: person_income mà chúng ta thấy có nhiều giá trị nhỏ và một vài giá trị rất lớn), việc sử dụng StandardScaler có thể nén phần lớn dữ liệu vào một khoảng nhỏ xung quanh mean, trong khi các outliers vẫn ở xa. Trong trường hợp này, MinMaxScaler có thể giữ được sự khác biệt tương đối giữa các điểm dữ liệu tốt hơn trong phạm vi $[0, 1]$.

Lựa chọn phương pháp: MinMaxScaler

- **Bảo toàn hình dạng phân phối:** MinMaxScaler co giãn dữ liệu về một phạm vi cố định (thường là $[0, 1]$) mà không làm thay đổi hình dạng phân phối ban đầu. Trong khi đó, StandardScaler cố gắng đưa dữ liệu về phân phối chuẩn (mean = 0, standard deviation = 1), điều này có thể làm thay đổi hình dạng của các phân phối lệch.
- **Ảnh hưởng của outliers:** Mặc dù chúng ta đã xử lý một số outliers, nhưng các phân phối lệch vẫn có thể chứa các giá trị ở đuôi có khoảng cách lớn so với phần lớn dữ liệu. StandardScaler nhạy cảm với outliers ở chỗ chúng có thể ảnh hưởng đến mean và standard deviation, dẫn đến việc phần lớn dữ liệu bị nén vào một khoảng nhỏ. MinMaxScaler ít bị ảnh hưởng hơn bởi các outliers sau khi chúng đã được giới hạn trong một phạm vi nhất định.
- **Phạm vi giá trị cố định:** MinMaxScaler đảm bảo rằng tất cả các đặc trưng sẽ có giá trị trong khoảng $[0, 1]$. Điều này có thể hữu ích cho một số thuật toán nhất định (ví dụ: neural networks) hoặc khi muốn so sánh trực tiếp tầm quan trọng tương đối của các đặc trưng.

Tóm lại, do sự hiện diện của các phân phối lệch đáng kể trong các cột số quan trọng như person_income, loan_amnt, loan_percent_income và cb_person_cred_hist_length, việc sử dụng MinMaxScaler có thể giúp bảo toàn tốt hơn sự khác biệt tỷ lệ giữa các giá trị trong một phạm vi cố định, đồng thời giảm thiểu tác động tiềm ẩn của các giá trị ngoại lai còn sót lại lên quá trình scaling, so với việc sử dụng StandardScaler có xu hướng thay đổi hình dạng phân phối và nhạy cảm hơn với outliers.

Các đặc trưng được lựa chọn để chuẩn hoá

Các cột được lựa chọn để chuẩn hoá là các đặc trưng dạng số thực (continuous features), loại trừ các biến đã được mã hoá nhị phân (như kết quả của One-Hot Encoding hoặc Label Encoding). Cụ thể bao gồm:



- **person_age**: Tuổi của người vay – đặc trưng số với quy mô nhỏ hơn đáng kể so với thu nhập hoặc khoản vay.
- **person_income**: Thu nhập cá nhân – có giá trị rất lớn và phân phối lệch, cần chuẩn hoá để đưa về thang đo tương đồng với các biến khác.
- **loan_amnt**: Số tiền vay – giá trị đa dạng và không có giới hạn cụ thể, cần chuẩn hoá để tránh ưu tiên các đặc trưng có độ lớn cao hơn.
- **loan_int_rate**: Lãi suất khoản vay – là một tỷ lệ phần trăm, có thể không đồng nhất về phân phối với **loan_percent_income**.
- **loan_percent_income**: Tỷ lệ giữa số tiền vay và thu nhập – đặc trưng dạng tỷ lệ, cũng dễ bị lệch phân phối.
- **cb_person_cred_hist_length**: Thời gian lịch sử tín dụng – tương tự với **person_age**, thường là một số năm dương, cần đưa về cùng phạm vi để tránh thiên lệch.

Model Test

Sau khi hoàn tất quá trình tiền xử lý dữ liệu, bao gồm việc xử lý outliers, mã hoá biến phân loại, và chuẩn hoá các đặc trưng dạng số thực bằng MinMaxScaler, chúng ta tiến hành huấn luyện và kiểm thử mô hình hồi quy logistic nhằm đánh giá hiệu quả của tập dữ liệu đã được xử lý.

Tập dữ liệu sau khi chuẩn hoá được chia thành hai phần: **80% để huấn luyện** và **20% để kiểm thử (validation)**. Việc phân chia này đảm bảo rằng mô hình không chỉ học thuộc dữ liệu huấn luyện mà còn có khả năng khái quát tốt trên dữ liệu chưa từng thấy.

Mô hình hồi quy logistic được lựa chọn vì đây là một mô hình tuyến tính đơn giản, dễ diễn giải và phù hợp với các bài toán phân loại nhị phân như đánh giá khả năng chấp thuận khoản vay.

Kết quả mô hình Logistic Regression với điểm F1-score đạt khoảng 56% phản ánh một số khía cạnh quan trọng về chất lượng tiền xử lý dữ liệu:

- **Ưu điểm**: Quá trình tiền xử lý đã thực hiện đầy đủ các bước như xử lý ngoại lệ, chuẩn hoá dữ liệu và mã hóa biến phân loại. Điều này giúp mô hình hoạt động ổn định, đạt được độ chính xác (accuracy) cao gần 90%, cho thấy dữ liệu đầu vào đã được làm sạch và đồng nhất đủ để tránh các lỗi tiềm ẩn trong quá trình huấn luyện.
- **Hạn chế**: Mặc dù accuracy cao, điểm recall chỉ đạt khoảng 45% và F1-score trung bình cho thấy mô hình vẫn còn bỏ sót một lượng đáng kể các trường hợp quan trọng, đặc biệt là các cá nhân có nguy cơ mất khả năng trả nợ. Đây có thể là dấu hiệu cho thấy dữ liệu vẫn còn mất cân bằng hoặc chưa thể hiện đầy đủ các đặc trưng quan trọng sau bước tiền xử lý hiện tại.



Bảng 4.3: Các chỉ số đánh giá mô hình Logistic Regression

Chỉ số	Giá trị
Accuracy	0.9007
Precision	0.7391
Recall	0.4526
F1-score	0.5614

Tổng quan

Bước tiền xử lý hiện tại có thể coi là đủ để xây dựng một mô hình cơ bản với hiệu năng ổn định, tuy nhiên chưa tối ưu để đạt hiệu quả phân loại cao nhất, đặc biệt trong các bài toán nhạy cảm như dự báo rủi ro tín dụng.

Để cải thiện hiệu suất, có thể cân nhắc các hướng sau:

- Sử dụng các phương pháp cân bằng dữ liệu (ví dụ: SMOTE, ADASYN) nhằm giảm thiểu ảnh hưởng của sự mất cân bằng lớp trong tập dữ liệu.
- Tăng cường bước kỹ thuật tạo đặc trưng (feature engineering) để khai thác thêm thông tin có giá trị từ dữ liệu gốc.
- Thử nghiệm các thuật toán phức tạp hơn hoặc tối ưu tham số mô hình để nâng cao khả năng dự báo.

4.3 Feature Engineering

4.3.1 Loại bỏ các đặc trưng không nhiều giá trị cho việc dự đoán

Feature	Mô tả
person_age	Tuổi của người vay
person_income	Tổng thu nhập hàng năm
person_emp_length	Số năm đi làm
loan_amnt	Số tiền vay
loan_int_rate	Lãi suất vay
loan_percent_income	Tỷ lệ phần trăm thu nhập dùng để trả nợ
cb_person_cred_hist_length	Lịch sử tín dụng (năm)
person_home_ownership	Tình trạng sở hữu nhà (RENT, OWN, MORTGAGE...)
loan_intent	Mục đích vay
loan_grade	Xếp hạng khoản vay (A-E)
cb_person_default_on_file	Từng vỡ nợ trong quá khứ (Y/N)
loan_status	Trạng thái vay (0 = Từ chối, 1 = Duyệt)

Bảng 4.4: Mô tả các đặc trưng trong dataset.

Dựa vào mô tả các đặc trưng trong dataset, các features này đều liên quan trực tiếp hoặc là gián tiếp cho việc dự đoán khả năng có thể được duyệt vay vì vậy chúng ta không có đặc trưng không liên quan đến đề tài để bỏ đi.



4.3.2 Tạo đặc trưng mới

affordability_score (Khả năng chi trả thực tế)

Để phản ánh sát hơn khả năng thực sự của người vay trong việc chi trả cho khoản vay (bao gồm cả phần lãi suất), chúng tôi xây dựng đặc trưng mới **affordability_score** với công thức:

$$\text{affordability_score} = \frac{\text{person_income}}{\text{loan_amnt} \times \left(\frac{\text{loan_int_rate}}{100} \right)}$$

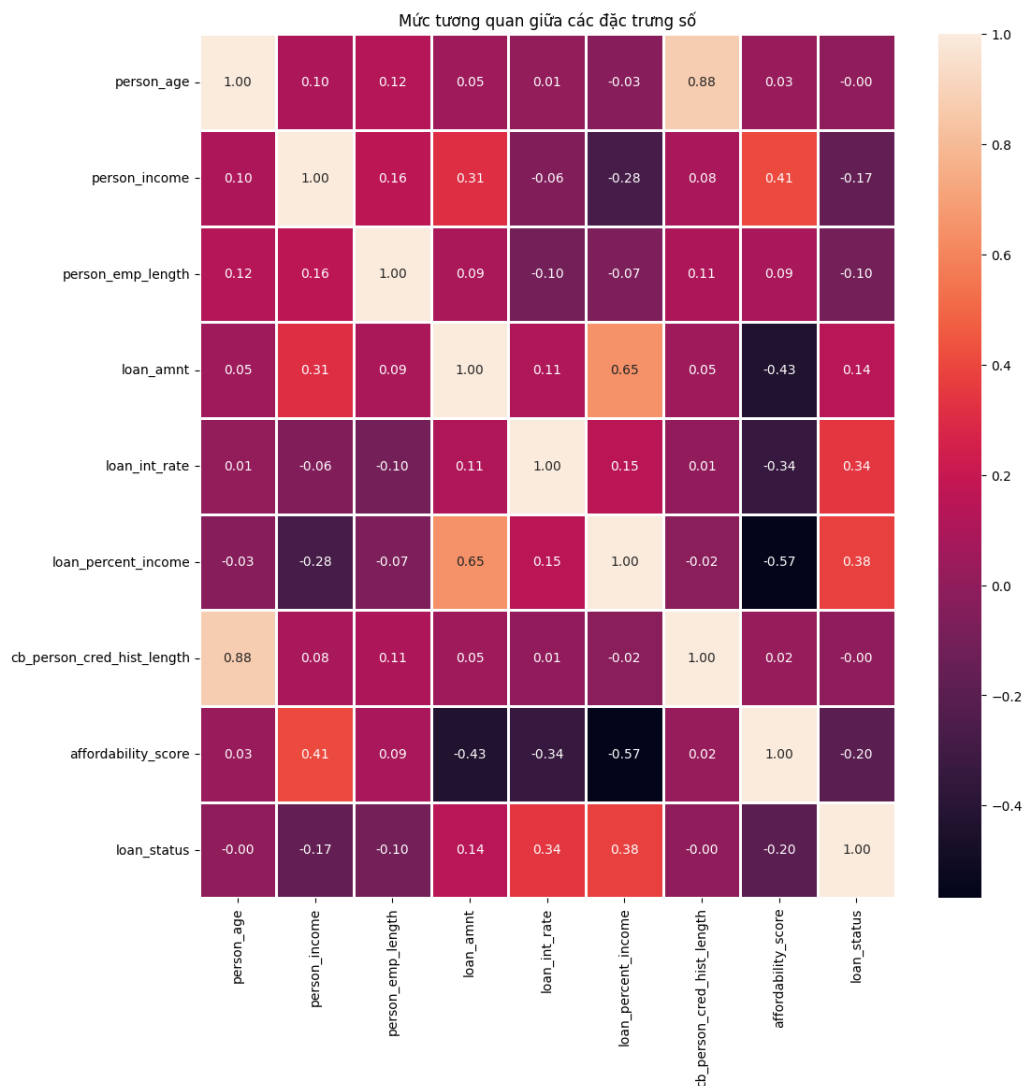
Cụ thể, chỉ số này được tính bằng cách lấy thu nhập hàng năm chia cho tổng chi phí lãi suất phải trả hàng năm. Khác với việc chỉ xét đến thu nhập hoặc khoản vay riêng lẻ, **affordability_score** kết hợp đồng thời ba yếu tố quan trọng: thu nhập hàng năm (`person_income`), số tiền vay (`loan_amnt`) và lãi suất vay (`loan_int_rate`).

- Đặc trưng này giúp mô hình hiểu rõ hơn về mức độ căng thẳng tài chính mà người vay phải đối mặt. Một người có thu nhập cao nhưng vay một khoản tiền lớn với lãi suất cao vẫn có thể gặp rủi ro, và đặc trưng này sẽ phản ánh được những trường hợp như vậy.
- Việc bổ sung **affordability_score** giúp mô hình nắm bắt tốt hơn mối liên hệ phức tạp giữa chi phí vay và khả năng chi trả, từ đó nâng cao độ chính xác khi dự đoán khả năng phê duyệt khoản vay.



4.3.3 Lựa chọn đặc trưng

Correlation matrix



Hình 4.11: Ma trận tương quan giữa các đặc trưng số

Hình 4.11 thể hiện hệ số tương quan Pearson giữa các đặc trưng số trong tập dữ liệu. Nhìn chung, dữ liệu có cấu trúc hợp lý với ít mối tương quan cao, giúp giảm nguy cơ dư thừa thông tin trong quá trình huấn luyện mô hình.

Hai đặc trưng tự tạo `affordability_score` và `emp_stability` cho thấy giá trị phân tách rõ rệt:

- `affordability_score` tương quan thuận với `person_income` (+0.41), nghịch với `loan_amnt` (-0.43), `loan_int_rate` (-0.34) và đặc biệt là `loan_percent_income` (-0.57) → thể hiện tốt khả năng chi trả của người vay.
- `emp_stability` tương quan mạnh với `person_emp_length` (+0.96), nhưng gần như độc lập với `person_age` (-0.09) → phù hợp để đánh giá sự ổn định nghề nghiệp mà không bị ảnh hưởng bởi tuổi tác.

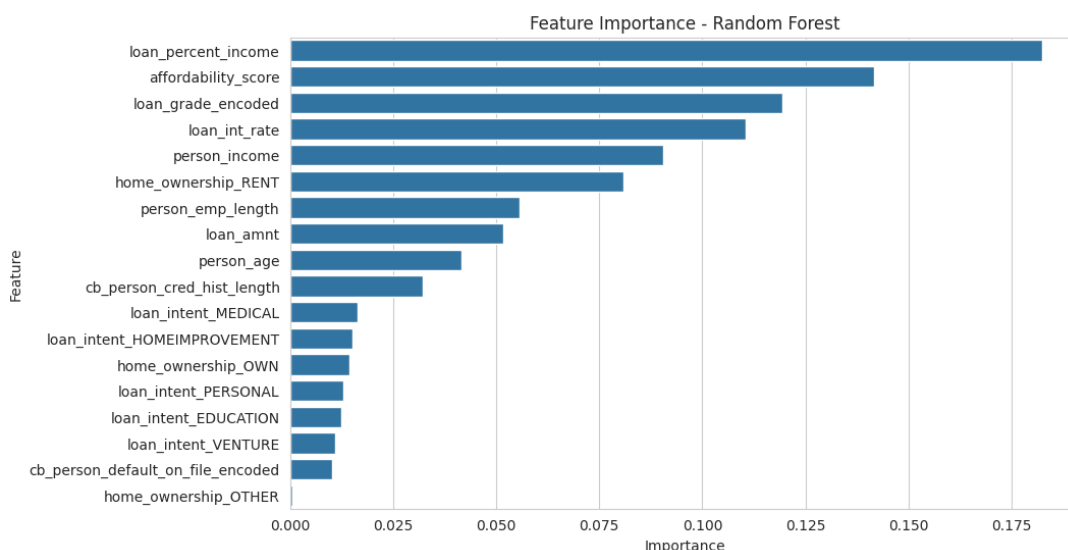


Một số mối liên hệ đáng chú ý khác:

- **person_age** và **cb_person_cred_hist_length** tương quan rất cao (+0.88) → cần cân nhắc khi sử dụng đồng thời trong mô hình tuyến tính.
- **loan_percent_income** liên hệ thuận với **loan_amnt** (+0.65), nghịch với **person_income** (−0.28) → thể hiện rõ mức gánh nặng tài chính.

Tổng thể, các đặc trưng được xây dựng mang lại thông tin giá trị, giúp tăng tính phân biệt và phù hợp để huấn luyện mô hình tín dụng.

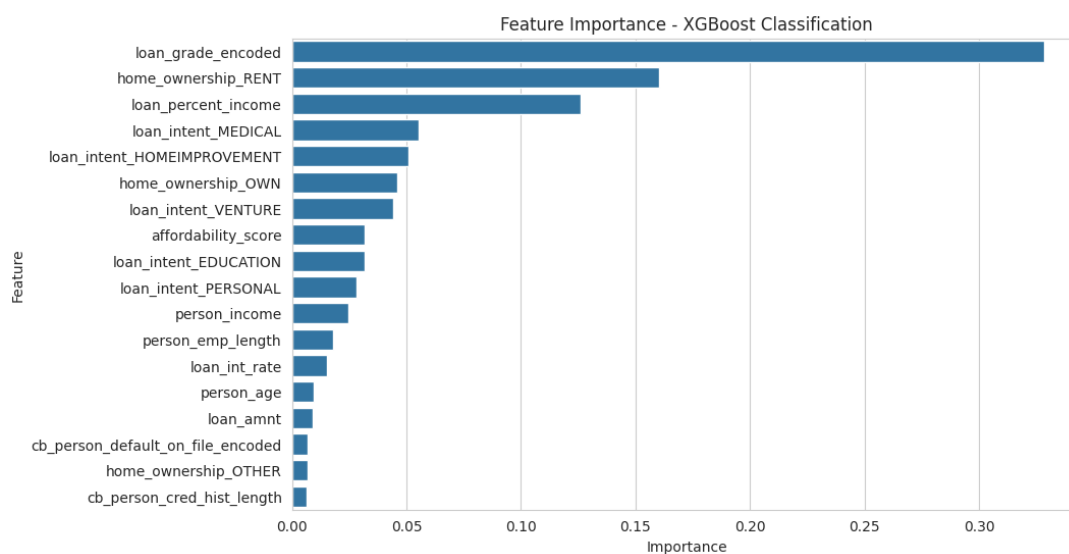
Model-based methods



Hình 4.12: Feature Importance từ mô hình Random Forest

Dựa trên biểu đồ 4.12, ta có thể rút ra một số đặc điểm quan trọng nhất đối với việc phân loại hồ sơ vay:

- **loan_percent_income** (tỷ lệ vay trên thu nhập) là biến quan trọng nhất, cho thấy ảnh hưởng lớn đến khả năng được duyệt vay.
- **affordability_score** và **loan_grade_encoded** cũng đóng vai trò lớn, phản ánh mức độ rủi ro và khả năng chi trả.
- Các đặc trưng như **loan_int_rate**, **person_income**, và **home_ownership_RENT** có ảnh hưởng đáng kể, cho thấy vai trò của mức thu nhập và hình thức sở hữu nhà.
- Các đặc trưng như **loan_intent** và **cb_person_default_on_file** ít ảnh hưởng hơn.



Hình 4.13: Feature Importance từ mô hình XGBoost

Kết quả từ mô hình XGBoost (hình 4.13) cho thấy một góc nhìn hơi khác về mức độ quan trọng:

- **loan_grade_encoded** là đặc trưng chiếm ưu thế rõ rệt, quan trọng hơn đáng kể so với các biến còn lại.
- **home_ownership_RENT** và **loan_percent_income** vẫn giữ vị trí cao trong danh sách.
- **loan_intent_MEDICAL**, **loan_intent_HOMEIMPROVEMENT**, và **loan_intent_VENTURE** có vai trò cao hơn so với đánh giá từ Random Forest.
- Các đặc trưng truyền thống như **person_income**, **loan_amnt** hay **loan_int_rate** lại có mức độ quan trọng thấp hơn.

Mặc dù hai mô hình có điểm tương đồng về một số đặc trưng chính như **loan_percent_income**, **affordability_score**, và **loan_grade_encoded**, tuy nhiên vẫn tồn tại sự khác biệt đáng kể trong cách đánh giá mức độ ảnh hưởng:

- **Random Forest** đánh giá cao hơn vai trò của các đặc trưng tài chính cụ thể như thu nhập, số tiền vay, lãi suất.
- **XGBoost** lại ưu tiên các đặc trưng phân loại như hạng vay, mục đích vay và tình trạng sở hữu nhà.

Từ kết quả này, chúng ta có thể sử dụng kết hợp các đặc trưng được đánh giá cao ở cả hai mô hình để tối ưu hóa hiệu suất mô hình hóa cuối cùng.



4.3.4 Kết luận

Việc lựa chọn đặc trưng đã được thực hiện thông qua kết hợp giữa kiểm định thống kê (Chi-Square và ANOVA) và đánh giá độ quan trọng từ các mô hình học máy như Random Forest và XGBoost. Mục tiêu là giữ lại những đặc trưng vừa mang ý nghĩa thống kê, vừa có ảnh hưởng đến hiệu quả dự đoán.

Bảng 4.5 dưới đây trình bày độ quan trọng của các đặc trưng đầu vào được sắp xếp theo thứ tự giảm dần:

Bảng 4.5: Độ quan trọng của các đặc trưng theo mô hình học máy

STT	Đặc trưng	Độ quan trọng
1	loan_percent_income	0.182
2	affordability_score	0.142
3	loan_grade_encoded	0.119
4	loan_int_rate	0.111
5	person_income	0.091
6	home_ownership_RENT	0.081
7	person_emp_length	0.056
8	loan_amnt	0.052
9	person_age	0.042
10	cb_person_cred_hist_length	0.032
11	loan_intent_MEDICAL	0.016
12	loan_intent_HOMEIMPROVEMENT	0.015
13	home_ownership_OWN	0.014
14	loan_intent_PERSONAL	0.013
15	loan_intent_EDUCATION	0.013
16	loan_intent_VENTURE	0.011
17	cb_person_default_on_file_encoded	0.010
18	home_ownership_OTHER	0.001

Nhận xét:

- Các đặc trưng có độ quan trọng cao nhất bao gồm:
 - loan_percent_income
 - affordability_score
 - loan_grade_encoded
- Một số đặc trưng có độ quan trọng thấp hơn 0.015, cụ thể:
 - loan_intent_VENTURE
 - cb_person_default_on_file_encoded
 - home_ownership_OTHER (chỉ 0.001)

Các đặc trưng này có thể được xem là yếu tố chỉ dựa vào tầm quan trọng trong mô hình. Tuy nhiên, chúng vẫn được giữ lại vì có ý nghĩa nghiệp vụ (ví dụ như mục đích vay hoặc trạng thái sở hữu nhà), giúp mô hình bao quát được những tình huống đặc biệt trong thực tế.



4.4 Huấn luyện, đánh giá và tinh chỉnh mô hình

4.4.1 Xác định Metric để evaluate model

Do tập dữ liệu mất cân bằng nghiêm trọng, chúng ta sẽ tập trung vào các chỉ số chính như F1-score, precision và recall, thay vì accuracy – chỉ được xem là chỉ số phụ. Lý do là accuracy dễ bị đánh giá sai lệch với dữ liệu không cân bằng, còn các chỉ số kia phản ánh chính xác hơn hiệu suất mô hình trên các lớp thiểu số

4.4.2 Chọn Baseline Model

Model	Precision	Recall	F1-Score	Accuracy
Logistic Regression	0.695	0.82	0.723	0.817
Random Forest	0.941	0.851	0.889	0.951

Bảng 4.6: Kết quả đánh giá hiệu suất của hai mô hình baseline.

Chúng tôi sẽ chọn 2 model để làm baseline đó chính là Logistic Regression và Random Forest và sử dụng K-Fold Validation và GridSearchCV để đánh giá hiệu suất. Dựa trên kết quả đánh giá, có thể thấy hiệu suất của Random Forest vượt trội hơn hẳn so với Logistic Regression, cho thấy các mô hình phù hợp với dữ liệu phi tuyến như Random Forest sẽ là lựa chọn tối ưu hơn cho bộ dữ liệu của bài toán này. Random Forest, với cấu trúc tập hợp từ nhiều cây quyết định, có khả năng nắm bắt các mối quan hệ phức tạp giữa các đặc trưng, dù tốn nhiều tài nguyên tính toán và khó diễn giải. Ngược lại, Logistic Regression là mô hình tuyến tính, đơn giản, dễ hiểu và nhanh trong huấn luyện, nhưng lại kém hiệu quả trên dữ liệu có mối quan hệ phi tuyến, dẫn đến kết quả không như mong đợi. Như vậy chúng ta sẽ đi tìm các mô hình thích hợp với dữ liệu phi tuyến.

4.4.3 Lựa Chọn Tiêu Chí và Mô Hình Phù Hợp

Bên cạnh việc tập dữ liệu bị mất cân bằng nghiêm trọng, số lượng mẫu trong dataset cũng rất lớn. Dựa trên kết quả từ các mô hình baseline, chúng tôi có thể xác định ba tiêu chí quan trọng để lựa chọn mô hình phù hợp cho bài toán này, bao gồm:

- Khả năng xử lý hiệu quả dữ liệu mất cân bằng.
- Phù hợp với tập dữ liệu có số lượng mẫu lớn.
- Tương thích với đặc tính phi tuyến của dữ liệu.

Qua đó, chúng tôi chọn ra được 3 models tiềm năng nhất đó chính là:

- XGBoost (Extreme Gradient Boosting)
- CatBoost (Categorical Boosting)
- LightGBM (Light Gradient Boosting Machine)



Việc chọn XGBoost, CatBoost và LightGBM cho bài toán này dựa trên sức mạnh của chúng trong việc xử lý dữ liệu phi tuyến và mất cân bằng. Cả ba đều thuộc nhóm gradient boosting, với XGBoost và LightGBM nổi bật về tốc độ trên tập dữ liệu lớn, còn CatBoost giỏi xử lý dữ liệu phân loại. Sự kết hợp này tận dụng điểm mạnh riêng để tối ưu hiệu suất.

4.4.4 Phương pháp tìm các siêu tham số tốt nhất cho các model

Chúng tôi sẽ sử dụng phương pháp StratifiedKFold Cross Validation(CV=5) để đánh giá hiệu suất mô hình một cách khách quan và đáng tin cậy, đảm bảo kết quả không bị thiên lệch bởi cách chia dữ liệu.

Để tìm ra các siêu tham số tối ưu cho từng mô hình, chúng tôi sẽ kết hợp chiến lược tìm kiếm thông minh cho các siêu tham số quan trọng của model: đầu tiên dùng GridSearchCV để xác định các khoảng giá trị khả thi, sau đó tận dụng Optuna để tinh chỉnh kỹ lưỡng hơn trong miền giá trị đã thu hẹp, nhằm đạt được hiệu quả tối đa. Bước dùng GridSearchCV chúng tôi sẽ làm ở ngoài nháp.

LightGBM

LightGBM (Light Gradient Boosting Machine) là một thư viện mã nguồn mở hiệu quả, được phát triển bởi Microsoft, tập trung vào thuật toán gradient boosting trên cây quyết định. Điểm nổi bật của LightGBM là tốc độ huấn luyện nhanh và khả năng xử lý tập dữ liệu lớn nhờ sử dụng kỹ thuật phân bin đặc trưng (histogram-based learning) và phát triển cây theo lá (leaf-wise growth), giúp giảm bộ nhớ và tăng hiệu suất. Thuật toán này rất phù hợp cho các bài toán phân loại và hồi quy, đặc biệt trong các tình huống cần tối ưu hóa tốc độ mà vẫn đảm bảo độ chính xác cao.

Siêu Tham Số	Phạm Vi Giá Trị	GridSearchCV
n_estimators	[200, 300, 400]	Đã lọc từ [100, 200, 300, 400, 500]
learning_rate	0.01 đến 0.3 (log scale)	Đã lọc từ [0.001, 0.5]
max_depth	7 đến 15	Đã lọc từ [3, 20]
num_leaves	63 đến 255 (step=16)	Đã lọc từ [31, 255]
min_child_samples	5 đến 20	Đã lọc từ [1, 30]
min_split_gain	0 đến 1	Giữ nguyên [0, 1]
subsample	0.6 đến 1.0	Giữ nguyên [0.6, 1.0]
colsample_bytree	0.6 đến 1.0	Giữ nguyên [0.6, 1.0]
reg_alpha	0.0 đến 10.0	Giữ nguyên [0.0, 10.0]
reg_lambda	0.0 đến 10.0	Giữ nguyên [0.0, 10.0]
scale_pos_weight	base_scale * 0.5 đến base_scale * 2.0	Giữ nguyên
random_state	42 (cố định)	Cố định
n_jobs	-1 (cố định)	Cố định

Bảng 4.7: Danh sách các siêu tham số của LightGBM



Siêu Tham Số	Giá Trị Tối Ưu
n_estimators	300
learning_rate	0.2078
max_depth	12
num_leaves	159
min_child_samples	15
min_split_gain	0.3602
subsample	0.9760
colsample_bytree	0.7487
reg_alpha	0.0606
reg_lambda	9.2280
scale_pos_weight	3.0203

Bảng 4.8: Siêu tham số tối ưu cho mô hình LightGBM.

	Precision	Recall	F1-Score	Support
Class 0	0.9633	0.9714	0.9673	50277
Class 1	0.8177	0.7762	0.7964	8324
Accuracy			0.9436	58601
Macro Avg	0.8905	0.8738	0.8819	58601
Weighted Avg	0.9426	0.9436	0.9430	58601

Bảng 4.9: Classification Report của mô hình LightGBM.



CatBoost

CatBoost (Categorical Boosting) là một thư viện mã nguồn mở mạnh mẽ, được phát triển bởi Yandex, tập trung vào thuật toán gradient boosting trên cây quyết định. Điểm nổi bật của CatBoost là khả năng xử lý dữ liệu phân loại (categorical data) một cách tự nhiên mà không cần mã hóa trước, cùng với hiệu suất cao trên các tập dữ liệu lớn nhờ tối ưu hóa tốc độ và giảm overfitting thông qua các kỹ thuật điều chuẩn. Thuật toán này rất phù hợp cho các bài toán phân loại và hồi quy, và được sử dụng rộng rãi trong các ứng dụng thực tế cũng như các cuộc thi học máy.

Siêu Tham Số	Phạm Vi Giá Trị	GridSearchCV
iterations	[200, 300, 400]	Đã lọc từ [100, 200, 300, 400, 500]
learning_rate	0.01 đến 0.3 (log scale)	Đã lọc từ [0.001, 0.5]
depth	4 đến 10	Đã lọc từ [2, 12]
l2_leaf_reg	1.0 đến 10.0	Giữ nguyên [1.0, 10.0]
bagging_temperature	0.0 đến 1.0	Giữ nguyên [0.0, 1.0]
border_count	32 đến 255	Đã lọc từ [16, 512]
random_strength	0.0 đến 1.0	Giữ nguyên [0.0, 1.0]
grow_policy	["SymmetricTree", "Depthwise", "Lossguide"]	Giữ nguyên
random_state	42 (cố định)	Cố định
verbose	0 (cố định)	Cố định
class_weights	class_weights (cố định)	Cố định

Bảng 4.10: Danh sách các siêu tham số của CatBoost.

Siêu Tham Số	Giá Trị Tối Ưu
iterations	400
learning_rate	0.1025
depth	10
l2_leaf_reg	5.8490
bagging_temperature	0.5590
border_count	255
random_strength	0.8743
grow_policy	Depthwise

Bảng 4.11: Siêu tham số tối ưu cho mô hình CatBoost sau quá trình tối ưu hóa Optuna.

	Precision	Recall	F1-Score	Support
Class 0	0.9616	0.9770	0.9692	50277
Class 1	0.8463	0.7643	0.8032	8324
Accuracy			0.9468	58601
Macro Avg	0.9040	0.8707	0.8862	58601
Weighted Avg	0.9452	0.9468	0.9457	58601

Bảng 4.12: Classification Report của mô hình CatBoost.



XGBoost

XGBoost (Extreme Gradient Boosting) là một thư viện mã nguồn mở mạnh mẽ, triển khai thuật toán gradient boosting trên cây quyết định. Nổi bật với tốc độ nhanh, hiệu suất cao nhờ tính toán song song và điều chuẩn chống overfitting, XGBoost phù hợp cho dữ liệu lớn, phức tạp và được sử dụng rộng rãi trong các bài toán phân loại, hồi quy cũng như các cuộc thi Kaggle. **Tính năng nổi bật của XGBoost:**

- **Tối ưu hóa hiệu suất cao:** Sử dụng tính toán song song và tối ưu hóa bộ nhớ để xử lý dữ liệu lớn
- **Regularization tích hợp:** Hỗ trợ L1 và L2 regularization để ngăn chặn overfitting
- **Xử lý missing values:** Tự động học cách xử lý giá trị thiếu trong quá trình training
- **Cross-validation tích hợp:** Cung cấp CV validation để đánh giá mô hình trong quá trình training
- **Feature importance:** Tự động tính toán và cung cấp độ quan trọng của từng feature
- **Early stopping:** Dừng training sớm khi performance không cải thiện thêm
- **Hỗ trợ đa nền tảng:** Tương thích với Python, R, Java, Scala và nhiều ngôn ngữ khác
- **GPU acceleration:** Hỗ trợ training trên GPU để tăng tốc độ xử lý
- **Flexible objective functions:** Cho phép tùy chỉnh loss function theo nhu cầu cụ thể
- **Scalability:** Xử lý được dataset có hàng tỷ samples với hiệu suất ổn định

Siêu Tham Số	Phạm Vi Giá Trị	GridSearch CV
n_estimators	[200, 300, 400]	Đã lọc từ [100, 200, 300, 400, 500]
learning_rate	0.05 đến 0.4 (log scale)	Đã lọc từ [0.005, 1.0]
max_depth	7 đến 18	Đã lọc từ [1, 25]
min_child_weight	7 đến 20	Đã lọc từ [1, 25]
gamma	0 đến 10	Giữ nguyên [0, 10]
subsample	0.5 đến 1.0	Giữ nguyên [0.5, 1.0]
colsample_bytree	0.5 đến 1.0	Giữ nguyên [0.5, 1.0]
reg_alpha	0.0 đến 10.0	Giữ nguyên [0.0, 10.0]
reg_lambda	0.0 đến 10.0	Giữ nguyên [0.0, 10.0]
scale_pos_weight	1.0 đến imbalance_ratio * 2.5	Giữ nguyên
use_label_encoder	False (cố định)	Không thay đổi
eval_metric	"logloss" (cố định)	Không thay đổi
random_state	42 (cố định)	Không thay đổi
verbosity	0 (cố định)	Không thay đổi

Bảng 4.13: Danh sách các siêu tham số của XGBoost



Siêu Tham Số	Giá Trị Tối Ưu
n_estimators	400
learning_rate	0.1769
max_depth	16
min_child_weight	19.4149
gamma	3.2823
subsample	0.5913
colsample_bytree	0.6792
reg_alpha	5.4764
reg_lambda	6.3202
scale_pos_weight	1.7553

Bảng 4.14: Siêu tham số tối ưu cho mô hình XGBoost sau quá trình tối ưu hóa Optuna.

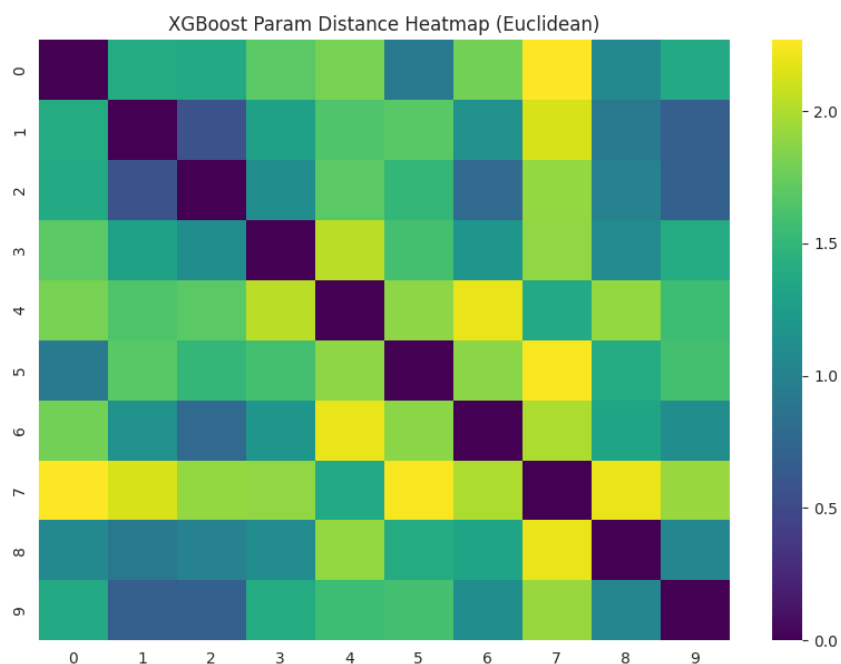
	Precision	Recall	F1-Score	Support
Class 0	0.9603	0.9825	0.9713	50277
Class 1	0.8770	0.7548	0.8113	8324
Accuracy			0.9501	58601
Macro Avg	0.9187	0.8686	0.8913	58601
Weighted Avg	0.9485	0.9501	0.9486	58601

Bảng 4.15: Classification Report của mô hình.

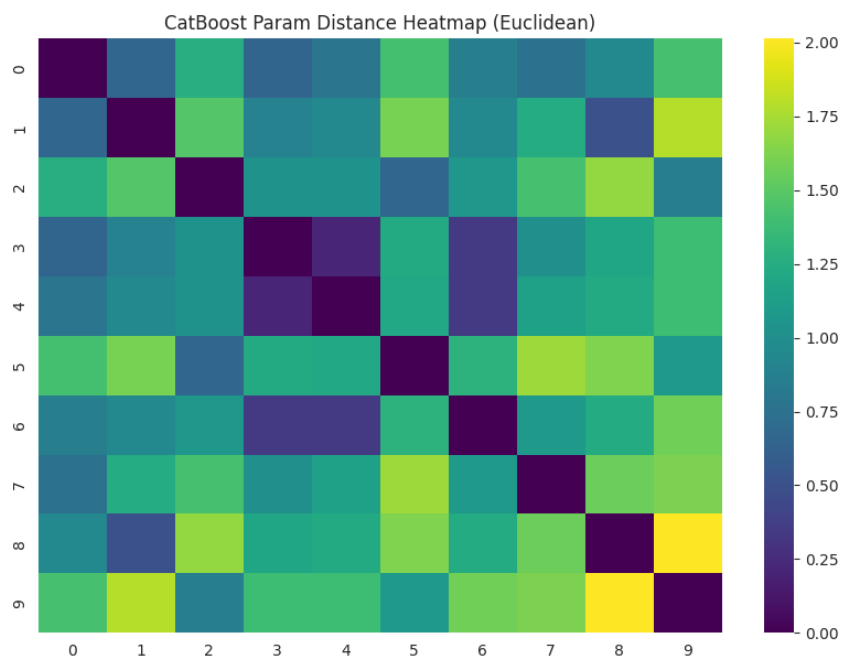
4.5 Ensemble Model

4.5.1 Check Độ Tương Đồng Các Bộ Siêu Tham Số Bằng Metric Euclidean

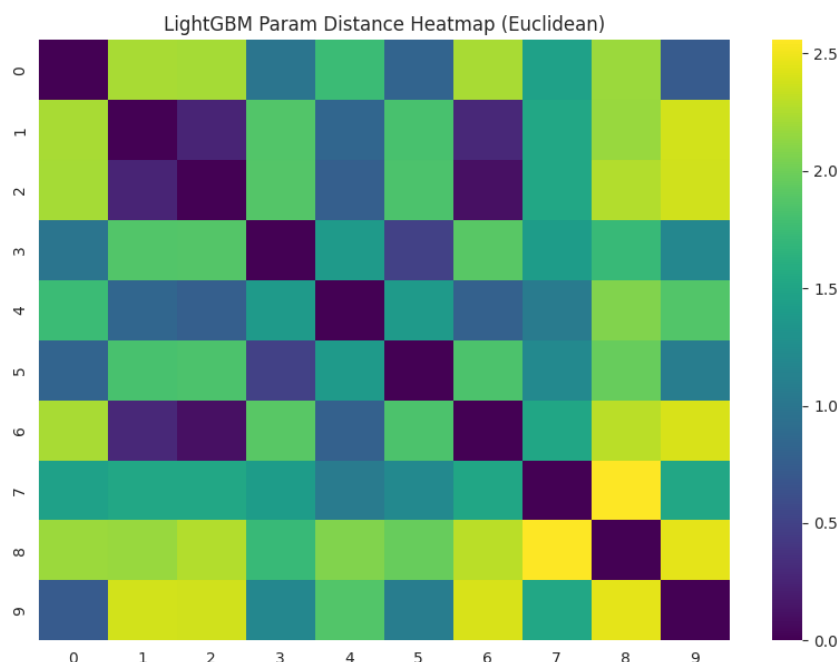
Trong phần này, chúng tôi sẽ đánh giá độ tương đồng giữa 10 bộ siêu tham số khác nhau của từng mô hình (XGBoost, CatBoost và LightGBM) bằng cách sử dụng metric Euclidean. Mục tiêu là đảm bảo rằng các bộ siêu tham số được chọn để ensemble không quá giống nhau, từ đó tăng tính đa dạng cho các mô hình kết hợp. Khoảng cách Euclidean được tính dựa trên các giá trị số hóa của các siêu tham số, giúp đo lường mức độ khác biệt giữa các bộ tham số trong cùng một mô hình. Phương pháp này cho phép chúng ta loại bỏ hoặc điều chỉnh các bộ siêu tham số quá gần nhau, đảm bảo hiệu quả của chiến lược ensemble. Dưới đây là các biểu đồ minh họa kết quả phân tích khoảng cách Euclidean giữa các bộ siêu tham số của từng mô hình:



Hình 4.14: Biểu đồ khoảng cách Euclidean giữa 10 bộ siêu tham số của mô hình XGBoost. Giá trị Eculidean lúc này là $0 < 1.4919 < \sqrt{10} \approx 3.162$



Hình 4.15: Biểu đồ khoảng cách Euclidean giữa 10 bộ siêu tham số của mô hình Catboost. Giá trị Eculidean lúc này là $0 < 1.1407 < \sqrt{10} \approx 3.162$



Hình 4.16: Biểu đồ khoảng cách Euclidean giữa 10 bộ siêu tham số của mô hình LightGBM.
Giá trị Euclidean lúc này là $0 < 1.5660 < \sqrt{10} \approx 3.162$

Dựa trên khoảng cách Euclidean giữa các mô hình, có thể thấy rằng các bộ siêu tham số không hoàn toàn giống nhau nhưng cũng không có sự khác biệt quá lớn. Nhìn chung, mức độ khác biệt này là chấp nhận được để tiến hành bước kết hợp mô hình (ensemble).

4.5.2 Train ensemble model

Đối với ba mô hình XGBoost, LightGBM và CatBoost, chúng tôi sẽ áp dụng các kỹ thuật ensemble phù hợp để tối ưu hóa hiệu suất trong tình huống này. Hai phương pháp phổ biến và hiệu quả nhất được lựa chọn là Voting và Stacking, nhằm tận dụng sức mạnh tổng hợp từ các mô hình riêng lẻ.

Soft Voting

Soft Voting là một kỹ thuật ensemble đơn giản nhưng hiệu quả, trong đó kết quả dự đoán cuối cùng được xác định dựa trên trung bình xác suất dự đoán của các mô hình riêng lẻ. Không giống như Hard Voting (lấy kết quả dựa trên số phiếu đa số), Soft Voting cân nhắc đến độ tin cậy của từng mô hình thông qua xác suất đầu ra, từ đó đưa ra dự đoán chính xác hơn, đặc biệt trong các bài toán phân loại với dữ liệu mất cân bằng. Ngoài ra, chúng tôi đánh trọng số xác suất của mỗi model khác nhau: XGBoost (0.4), CatBoost (0.3), LightGBM (0.3).

Trong phần này, chúng tôi sử dụng kỹ thuật **Out-of-Fold Predictions (OOF)** để tạo ra xác suất dự đoán đại diện cho từng mô hình, đảm bảo đánh giá chính xác và hạn chế overfitting trong quá trình huấn luyện. Soft Voting được triển khai theo hai hướng tiếp cận sau:

Dùng OOF với 1 Bộ Siêu Tham Số Tốt Nhất của 3 Models: XGBoost, CatBoost, LightGBM



Trong cách tiếp cận đầu tiên, chúng tôi sử dụng **1 bộ siêu tham số tốt nhất** của từng mô hình (XGBoost, CatBoost và LightGBM) đã được tìm ra thông qua Optuna. Với mỗi mô hình, chúng tôi tiến hành huấn luyện theo K -Fold và tạo ra xác suất OOF dự đoán cho toàn bộ tập huấn luyện. Xác suất OOF trung bình được tính riêng cho từng mô hình, sau đó áp dụng Soft Voting với trọng số đã nêu.

Phương pháp này không chỉ tận dụng hiệu suất tốt nhất của từng mô hình mà còn đảm bảo rằng dự đoán không bị thiên lệch do overfitting, vì mỗi điểm dữ liệu chỉ được dự đoán bởi mô hình không thấy nó trong quá trình huấn luyện.

	Precision	Recall	F1-Score	Support
Class 0	0.9593	0.9853	0.9721	50277
Class 1	0.8937	0.7476	0.8142	8324
Accuracy			0.9515	58601
Macro Avg	0.9265	0.8664	0.8931	58601
Weighted Avg	0.9500	0.9515	0.9497	58601

Bảng 4.16: Báo cáo phân loại sử dụng OOF + Soft Voting với 1 bộ siêu tham số mỗi mô hình.

Dùng OOF với 10 Bộ Siêu Tham Số Tốt Nhất của 3 Models: XGBoost, CatBoost, LightGBM

Trong cách tiếp cận thứ hai, chúng tôi mở rộng bằng cách sử dụng **10 bộ siêu tham số tốt nhất** cho từng mô hình (XGBoost, CatBoost, LightGBM). Mỗi bộ siêu tham số sinh ra một tập xác suất OOF độc lập. Sau đó, chúng tôi tính trung bình xác suất OOF trên 10 bộ này để tạo ra một xác suất tổng hợp cho từng mô hình. Cuối cùng, áp dụng Soft Voting với trọng số tương tự.

Phương pháp này làm tăng tính đa dạng trong dự đoán, giảm rủi ro bị ảnh hưởng bởi một bộ siêu tham số cụ thể, và có thể giúp tăng độ tổng quát (generalization) của mô hình ensemble.

	Precision	Recall	F1-Score	Support
Class 0	0.9590	0.9862	0.9724	50277
Class 1	0.8997	0.7456	0.8154	8324
Accuracy			0.9520	58601
Macro Avg	0.9294	0.8659	0.8939	58601
Weighted Avg	0.9506	0.9520	0.9501	58601

Bảng 4.17: Báo cáo phân loại sử dụng OOF + Soft Voting với 10 bộ siêu tham số mỗi mô hình.



Stacking

Stacking (Stacked Generalization) là một kỹ thuật ensemble tiên tiến, trong đó các dự đoán từ nhiều mô hình cơ sở như XGBoost, CatBoost và LightGBM được sử dụng làm đầu vào cho một mô hình meta để đưa ra kết quả cuối cùng. Chúng tôi sử dụng Logistic Regression làm meta-model nhờ tính đơn giản và khả năng diễn giải, giúp kết hợp hiệu quả các dự đoán mà không gây overfitting. Phương pháp này cho phép tận dụng điểm mạnh của từng mô hình, nâng cao độ chính xác trong các bài toán phân loại phức tạp.

	Precision	Recall	F1-Score	Support
Class 0	0.9631	0.9782	0.9706	50277
Class 1	0.8548	0.7735	0.8121	8324
Accuracy			0.9492	58601
Macro Avg	0.9089	0.8759	0.8914	58601
Weighted Avg	0.9477	0.9492	0.9481	58601

Bảng 4.18: Báo cáo phân loại (Classification Report) của mô hình.

Siêu Tham Số	Giá Trị	Mô Tả
random_state	42	Cố định seed cho reproducibility
class_weight	'balanced'	Cân bằng trọng số các class
max_iter	1000	Số lần lặp tối đa
C	0.00001	Tham số regularization nghịch đảo

Bảng 4.19: Siêu tham số của Meta Learner (Logistic Regression).

4.6 Kết quả thực nghiệm

Bảng 4.20 trình bày tổng quan hiệu suất của các mô hình đã huấn luyện, bao gồm LightGBM, CatBoost, XGBoost và mô hình Ensemble được xây dựng từ ba mô hình trên. Hai chỉ số đánh giá chính được sử dụng là **F1 Score** và **Accuracy (ACC)**.

- **LightGBM:** đạt F1 Score là 0.8819 và Accuracy là 94.36%. Đây là mô hình có tốc độ huấn luyện nhanh, hiệu quả trên dữ liệu lớn và xử lý tốt missing values.
- **CatBoost:** đạt F1 Score là 0.8862 và Accuracy là 94.68%. Dù có độ chính xác thấp hơn một chút so với LightGBM, nhưng CatBoost lại có ưu thế trong xử lý dữ liệu dạng phân loại mà không cần encode phức tạp.
- **XGBoost:** nổi bật hơn với F1 Score 0.8913 và Accuracy 95.01%, cho thấy khả năng dự đoán ổn định và hiệu quả, đặc biệt phù hợp với bài toán phức tạp nhiều đặc trưng như dự đoán phê duyệt khoản vay.
- **Mô hình Ensemble:** có hiệu suất vượt trội với F1 Score đạt 0.8940 và Accuracy lên đến 95.2%. Đây là kết quả tốt nhất trong số các mô hình, cho thấy việc kết hợp (ensemble) giúp tận dụng được điểm mạnh của từng mô hình thành phần và giảm thiểu sai số riêng lẻ.



Nhìn chung, việc sử dụng mô hình Ensemble đã mang lại hiệu quả đáng kể về cả độ chính xác và khả năng tổng quát hóa mô hình, nhờ vào chiến lược phối hợp đầu ra của các mô hình mạnh khác nhau.

Bảng 4.20: Tổng quan Hiệu suất của các Mô hình

Mô hình	F1 Score	Accuracy (ACC)
LightGBM	0.8819	0.9436
CatBoost	0.8862	0.9468
XGBoost	0.8913	0.9501
Ensemble	0.8940	0.9520

Chương 5

KẾT LUẬN VÀ HƯỚNG PHÁT TRIỂN

5.1 Kết luận

Trong khuôn khổ đề án này, nhóm đã tiếp cận bài toán dự đoán khả năng được phê duyệt vay vốn dựa trên dữ liệu khách hàng với quy trình xử lý và xây dựng mô hình hoàn chỉnh. Thông qua quá trình phân tích dữ liệu (EDA), xử lý dữ liệu thiếu và ngoại lai, mã hóa và chuẩn hóa dữ liệu, nhóm đã tạo ra một tập dữ liệu sạch và tối ưu cho việc huấn luyện mô hình. Sau đó, ba mô hình máy học mạnh mẽ là **CatBoostClassifier**, **LGBMClassifier** và **XGBClassifier** đã được triển khai, huấn luyện và tinh chỉnh thông qua kỹ thuật *cross-validation* và *GridSearchCV*.

Kết quả cuối cùng được cải thiện bằng cách sử dụng kỹ thuật **Voting Ensemble**, giúp tận dụng ưu điểm của từng mô hình riêng lẻ để nâng cao độ chính xác tổng thể. Quá trình đánh giá mô hình chủ yếu dựa trên chỉ số **F1-score** – một metric quan trọng phản ánh khả năng phân biệt giữa hai lớp. Các biểu đồ, chỉ số và kết quả phân tích đã chứng minh rằng pipeline đề xuất có khả năng hoạt động hiệu quả và ổn định trên tập dữ liệu cho trước.

5.2 Hướng phát triển

Mặc dù pipeline hiện tại đã đạt được kết quả khả quan, nhóm nhận thấy vẫn còn một số hướng đi tiềm năng trong tương lai:

- **Thử nghiệm với các mô hình mới:** Trong tương lai, nhóm có thể thử nghiệm thêm các mô hình tiên tiến hơn như *TabNet*, *Node* hoặc *DeepGBM* nhằm khai thác tốt hơn mối quan hệ phức tạp trong dữ liệu dạng bảng.
- **Tối ưu hóa đặc trưng nâng cao:** Áp dụng kỹ thuật *feature selection* tự động như *Boruta*, *SHAP-based importance*, hoặc *AutoFeat* để phát hiện và loại bỏ các đặc trưng dư thừa, giúp giảm độ phức tạp của mô hình.
- **Ứng dụng thực tế:** Tích hợp mô hình vào hệ thống đánh giá tín dụng tự động cho các ngân hàng hoặc tổ chức tài chính nhằm kiểm tra khả năng hoạt động trong môi trường thực tế.
- **Tăng cường tính giải thích mô hình:** Sử dụng các công cụ như *LIME*, *SHAP*, hoặc *ELI5* để lý giải quyết định của mô hình, giúp tăng tính minh bạch và tin cậy trong các ứng dụng tài chính.



- **Học tăng cường dữ liệu:** Áp dụng các kỹ thuật như *SMOTE*, *ADASYN* hoặc *Data Augmentation* chuyên biệt cho dữ liệu bảng để cân bằng lớp hoặc tăng cường dữ liệu đầu vào.

Nhóm tin rằng với những hướng phát triển này, hệ thống dự đoán phê duyệt khoản vay sẽ ngày càng trở nên hiệu quả, đáng tin cậy và có khả năng mở rộng trong các ứng dụng thực tiễn.

Chương 6

BẢNG PHÂN CÔNG CÔNG VIỆC

Họ và tên	MSSV	Đóng góp (%)	Vai trò/Phần việc đảm nhận
Huỳnh Việt Tiến	23521570	32.5%	Leader, thiết kế slide, trình bày, preprocessing, EDA
Nguyễn Minh Bảo	23520123	32.5%	Trình bày, build model & tuning, EDA, Feature engineering
Nguyễn Công Phát	23521143	25%	format notebook & report, EDA, feature engineering
Phạm Phú Bảo	23520133	10%	EDA, slide
<i>Tổng cộng: 100%.</i>			

Bảng 6.1: Phân chia công việc và mức đóng góp của các thành viên trong nhóm

Mức chia điểm trên đã được cả nhóm thống nhất sau khi trao đổi và đánh giá khách quan.

Tài liệu tham khảo

- [CG16] Tianqi Chen and Carlos Guestrin. “XGBoost: A Scalable Tree Boosting System”. In: **Proceedings of the 22nd ACM SIGKDD International Conference on Knowledge Discovery and Data Mining**. KDD '16. ACM, Aug. 2016. doi: 10.1145/2939672.2939785.
- [Ke+17] Guolin Ke et al. “LightGBM: A Highly Efficient Gradient Boosting Decision Tree”. In: **Neural Information Processing Systems**. 2017. <https://api.semanticscholar.org/CorpusID:3815895>.
- [Pro+19] Liudmila Prokhorenkova et al. “CatBoost: unbiased boosting with categorical features”. 2019. arXiv:1706.09516 [cs.LG].
- [GOV22] Léo Grinsztajn, Edouard Oyallon, and Gaël Varoquaux. “Why do tree-based models still outperform deep learning on tabular data?”. 2022. arXiv:2207.08815 [cs.LG].
- [BHT23] Stephen Bates, Trevor Hastie, and Robert Tibshirani. “Cross-Validation: What Does It Estimate and How Well Does It Do It?”. In: **Journal of the American Statistical Association** (May 2023), pp. 1–12. doi: 10.1080/01621459.2023.2197686.